Introduction au logiciel 

Support de cours et d'exercices

I. C. GERALDO

Issa Cherif GERALDO

Novembre 2024

TABLE DES MATIÈRES

0 Note introductive **5**
- 0.1 Bref descriptif du cours . . . 5
- 0.2 Objectif général . . . 5
- 0.3 Structure du document . . . 5

1 Généralités sur le logiciel R **6**
- 1.1 Présentation . . . 6
- 1.2 Installation . . . 7
  - 1.2.1 Installation sous Windows . . . 7
  - 1.2.2 Installation sous Linux . . . 8
- 1.3 Lancement et fermeture de R . . . 8
  - 1.3.1 Lancement de R sous Windows . . . 8
  - 1.3.2 Lancement de R sous Linux . . . 10
  - 1.3.3 Fermeture de R . . . 10
- 1.4 Commandes élémentaires . . . 11
  - 1.4.1 Création de variables et affichage . . . 11
  - 1.4.2 Séparation de commandes . . . 12
  - 1.4.3 Espace de travail . . . 12
  - 1.4.4 Documentation de R . . . 13
  - 1.4.5 Commentaires . . . 13
- 1.5 Méthodes d'exécution des commandes . . . 13
  - 1.5.1 Première méthode . . . 13
  - 1.5.2 Deuxième méthode . . . 14
- 1.6 L'environnement de développement intégré RStudio . . . 15
- 1.7 Bibliothèques ou packages . . . 16

2 Classes d'objets et fonctions usuelles **18**
- 2.1 Introduction . . . 19
- 2.2 Vecteurs de nombres réels (classe `numeric`) et d'entiers relatifs (classe `integer`) 19
  - 2.2.1 Valeurs particulières . . . 19
  - 2.2.2 Création . . . 19
  - 2.2.3 Opérations et fonctions . . . 21
  - 2.2.4 Composantes . . . 24
  - 2.2.5 Opérations entières . . . 24
  - 2.2.6 Nommer les composantes d'un vecteur . . . 24
- 2.3 Vecteurs de nombres complexes (classe `complex`) . . . 25

I.C. GERALDO

2.4 Matrices (classe `matrix`) . . . 26
2.4.1 Création et accès aux composantes . . . 26
2.4.2 Opérations et fonctions matricielles . . . 27
2.4.3 Nommer les lignes et colonnes d'une matrice . . . 29
2.5 Vecteurs de chaînes de caractères (classe `character`) et facteurs (classe `factor`) 30
2.5.1 Vecteurs de chaînes de caractères (classe `character`) . . . 30
2.5.2 Facteurs (classe `factor`) . . . 32
2.6 Listes (classe `list`) . . . 33
2.7 Tableaux de données (classe `data.frame`) . . . 33
2.7.1 Création par lignes de commandes . . . 33
2.7.2 Jeux de données internes . . . 34
2.7.3 Importation de tableaux de données . . . 35
2.7.4 Manipulation . . . 36
2.8 Vecteurs logiques (classe `logical`) . . . 39
2.8.1 Création par saisie . . . 39
2.8.2 Opérations sur les vecteurs logiques . . . 39
2.8.3 Opérateurs de comparaison . . . 40
2.8.4 Opérateurs (connecteurs) logiques . . . 40
2.8.5 Fonctions logiques usuelles . . . 41
2.8.6 Sélection d'éléments dans un vecteur . . . 42
2.8.7 Sélection d'éléments dans un tableau (matrice ou tableau de données) 42
2.9 Exercices . . . 44

3 Statistique descriptive 48
3.1 Séries statistiques à une variable . . . 48
3.1.1 Variable quantitative . . . 48
3.1.2 Variables qualitatives . . . 58
3.2 Tableaux de contingence à double entrée . . . 60
3.2.1 Généralités . . . 60
3.2.2 Distributions et fréquences marginales . . . 61
3.2.3 Profils lignes et profils colonnes . . . 61
3.2.4 Représentation graphique . . . 62
3.3 Ajustement linéaire . . . 64
3.3.1 Nuage de points . . . 64
3.3.2 Coefficient de corrélation linéaire . . . 65
3.3.3 Ajustement linéaire . . . 65
3.4 Exercices . . . 68

4 Programmation dans R 70
4.1 Structures de contrôle . . . 70
4.1.1 Exécution conditionnelle avec `if` . . . 70
4.1.2 Boucle `for` . . . 72
4.1.3 Boucles `while` et `repeat` . . . 73
4.2 Création de nouvelles fonctions . . . 75
4.2.1 Création . . . 75

I. C. GERALDO

4.2.2 Appel de la fonction . . . 75
4.2.3 Affichage du code et des arguments d'une fonction . . . 75
4.2.4 Sorties multiples . . . 76
4.2.5 Arguments par défaut . . . 76
4.2.6 Gestion des erreurs sur les arguments . . . 76
4.2.7 Fonctions vectorisées (ou vectorielles) . . . 77
4.3 Exercices . . . 79

5 Fonctionnalités graphiques **80**
5.1 Généralités . . . 80
5.2 Gestion de la fenêtre graphique . . . 81
5.2.1 Création et fermeture . . . 81
5.2.2 Fractionnement d'une fenêtre graphique . . . 81
5.2.3 Sauvegarde d'une fenêtre graphique dans un fichier . . . 83
5.3 Fonctions haut niveau . . . 83
5.3.1 La fonction `plot` . . . 84
5.3.2 Autres fonctions haut niveau . . . 85
5.4 Fonctions bas niveau . . . 85
5.4.1 Ajout de points, de lignes ou de symboles . . . 86
5.4.2 Ajout de texte . . . 87
5.4.3 Ajout d'un quadrillage sur le graphique . . . 87
5.4.4 Gestion des axes . . . 87
5.5 Fonction `curve` : une fonction à la fois haut niveau et bas niveau . . . 88
5.6 Exercices . . . 89

6 Probabilités, intégrales et simulations **91**
6.1 Lois de probabilités usuelles . . . 91
6.2 Tirages avec ou sans remise . . . 92
6.3 Réplication d'une commande . . . 93
6.4 Calcul d'intégrales . . . 93
6.5 Exercices . . . 95

Références bibliographiques **97**

I. C. GERALDO

CHAPITRE 0

NOTE INTRODUCTIVE

0.1 BREF DESCRIPTIF DU COURS

Ce cours présente le logiciel statistique R et son utilisation pour mettre en œuvre les méthodes et techniques usuelles en statistique descriptive et en calcul de probabilités. Il peut être utile à toute personne ayant suivi les cours de statistique descriptive et de calcul de probabilités (niveau première année de Licence) et souhaitant appliquer ces cours à l'aide d'un logiciel statistique.

0.2 OBJECTIF GÉNÉRAL

L'objectif général de ce cours est d'appliquer les méthodes usuelles de la statistique descriptive et du calcul de probabilités avec le logiciel statistique R.

A la fin de ce cours, l'apprenant devra être capable de :

- saisir des données dans le logiciel R (chapitres 1 et 2) ;
- appliquer les méthodes de la statistique descriptive à l'aide du logiciel R (chapitre 3) ;
- programmer en R (chapitre 4) ;
- faire des représentations graphiques (chapitre 5) ;
- implémenter quelques scénarios probabilistes classiques (calcul de probabilités pour les lois usuelles, lancer de dés, simulations simples, etc. . .) (chapitre 6).

J. C. GERALDO

0.3 STRUCTURE DU DOCUMENT

Le présent document est subdivisé en six (06) chapitres contenant chacun des syntaxes et des exemples détaillés. Chaque chapitre finit par des exercices non corrigés.

Il est vivement conseillé au lecteur d'installer le logiciel R et de tester les syntaxes et les exemples. Le lecteur est aussi exhorté à consulter la documentation de R pour une compréhension plus détaillée des commandes.

CHAPITRE 1

GÉNÉRALITÉS SUR LE LOGICIEL R

Sommaire

1.1 Présentation **6**
1.2 Installation **7**
1.2.1 Installation sous Windows 7
1.2.2 Installation sous Linux 8
1.3 Lancement et fermeture de R **8**
1.3.1 Lancement de R sous Windows 8
1.3.2 Lancement de R sous Linux 10
1.3.3 Fermeture de R 10
1.4 Commandes élémentaires **11**
1.4.1 Création de variables et affichage 11
1.4.2 Séparation de commandes 12
1.4.3 Espace de travail 12
1.4.4 Documentation de R 13
1.4.5 Commentaires 13
1.5 Méthodes d'exécution des commandes **13**
1.5.1 Première méthode 13
1.5.2 Deuxième méthode 14
1.6 L'environnement de développement intégré RStudio **15**
1.7 Bibliothèques ou packages **16**

J.C.GERALDO

1.1 PRÉSENTATION

Le logiciel R est un logiciel d'analyse statistique et graphique créé par deux statisticiens Ross Ihaka (statisticien néo-zélandais) et Robert Gentleman (statisticien canadien) à l'université d'Auckland (Nouvelle-Zélande) dans le cadre d'un projet de recherche débuté en 1993.

Il est un clone gratuit du logiciel payant `S-plus`, dérivé du langage `S` (Logiciel payant) développé dans les années 1970 par une équipe de chercheurs menée par un statisticien du nom de John M. Chambers. Depuis 1997, son développement et sa distribution sont assurés par plusieurs statisticiens rassemblés dans le **R Development Core Team**.

Le logiciel R présente plusieurs avantages : c'est un logiciel **libre** et **gratuit**, très complet, en essor permanent, disponible pour divers environnements : Linux, Windows, Macintosh. C'est un logiciel utilisé en entreprise, dans le monde académique et de la recherche, au sein d'organismes publics ou d'ONG ou encore par les analystes (data miners, data scientists) spécialistes de la fouille de données.

Le site officiel du logiciel R est :

https://www.r-project.org/

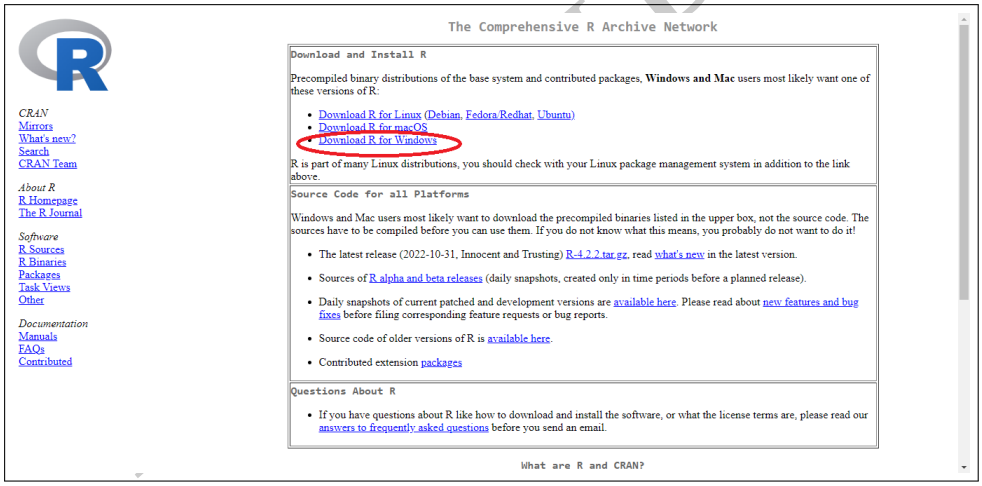
1.2 INSTALLATION

1.2.1 Installation sous Windows

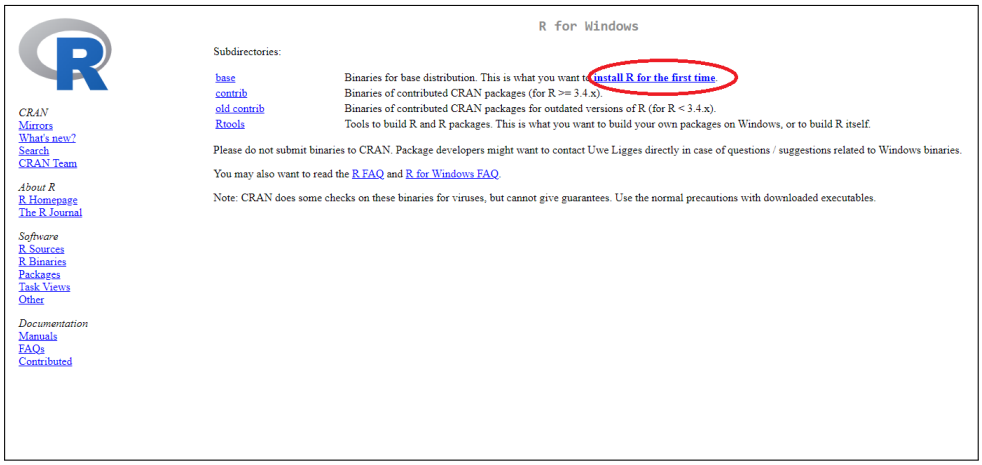
Pour télécharger le fichier d'installation, il faut :

- se rendre sur le site du CRAN (Comprehensive R Archive Network) sur lequel se trouvent les ressources pour le logiciel R : https://cran.r-project.org/)

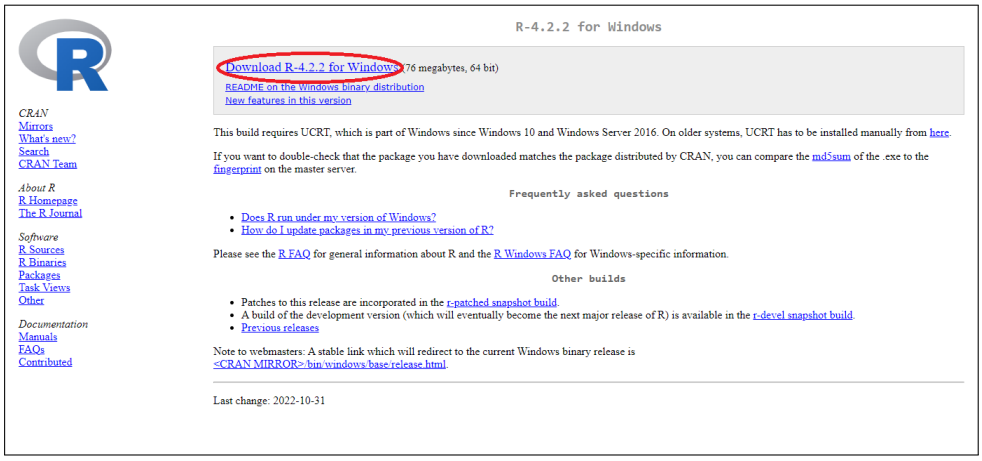
- cliquer sur le lien **Download R for Windows** :



- cliquer sur **install R for the first time** :



- cliquer sur **Download R-x.x.x for Windows**.



Après téléchargement du fichier d'installation, il faut l'exécuter par double-clic. **Durant l'installation, il ne faut pas oublier de cocher l'option de mettre un raccourci sur le bureau**. Une fois l'installation terminée, le logiciel R est accessible (comme tout autre logiciel) dans le menu *Démarrer* ou dans *Tous les programmes* ou encore sur le bureau. Sur le bureau, il est repérable par son icône :



I. C. GERALDO

1.2.2 Installation sous Linux

Elle peut se faire via le *gestionnaire des paquets* en installant les paquets `r-base-dev` et `r-base`.

1.3 Lancement et fermeture de R

1.3.1 Lancement de R sous Windows

Pour lancer R **sous Windows**, il suffit de double-cliquer sur l'icône



Il s'ouvre alors une interface graphique appelée RGUI (R Graphical User Interface) comme le montre la figure 1.1 (page 9).

Cette interface graphique contient une fenêtre de la forme de la figure 1.2 (page 9) appelée **console de R**.

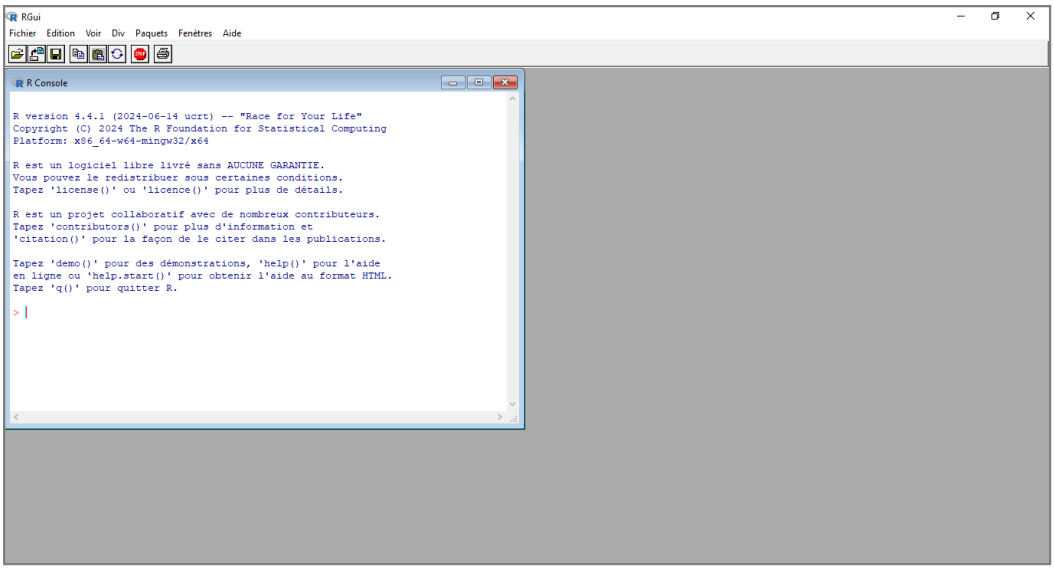


FIGURE 1.1 – Aperçu de RGUI (R Graphical User Interface).

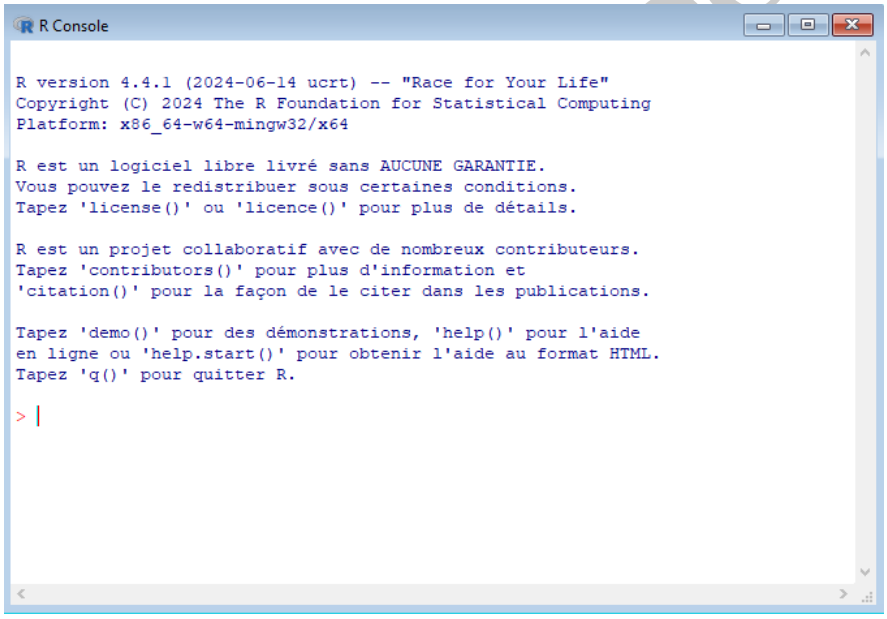


FIGURE 1.2 – Aperçu de la console de R au lancement de R. Le symbole "**>**", appelé prompt, indique que R est prêt attend une commande.

```
>
```

Le symbole "**>**", appelé **prompt**, indique que R est prêt et attend une commande.

Remarque 1.1 Lorsque **le symbole "+" apparaît au lieu de ">"**, cela signifie que la commande en cours de saisie est incomplète (par exemple, il manque une parenthèse ou une accolade) et que R attend qu'elle soit complétée. On peut alors :

- soit compléter la commande et appuyer sur la touche **Enter** ↩ pour l'exécuter,

- soit appuyer sur la touche **Esc** ou **Echap** du clavier pour annuler la commande et revenir à >.

Remarque 1.2 On peut vider la console à tout moment en appuyant simultanément sur les touches **Ctrl** et **L**.

1.3.2 Lancement de R sous Linux

Sous Linux, l'on peut lancer R depuis un terminal Linux en tapant simplement la commande R dans le terminal (voir la figure 1.3, page 10 et la figure 1.4, page 11).

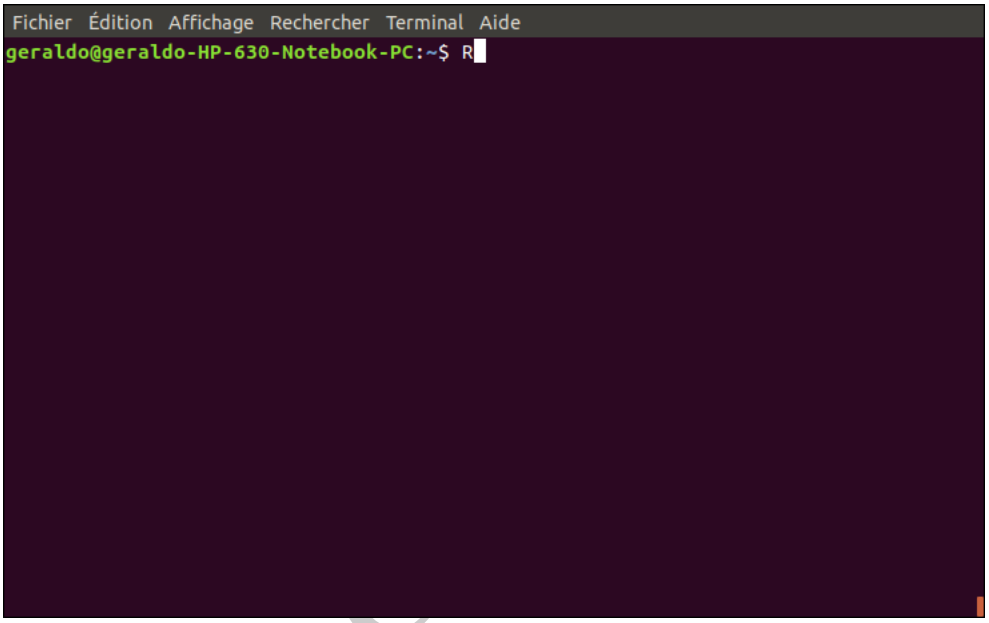


FIGURE 1.3 – Terminal Linux avant lancement de R

1.3.3 Fermeture de R

```
quit()
q()
```

Commandes équivalentes à saisir dans la console pour fermer R. Bien que vide, la paire de parenthèses est indispensable ici car il s'agit d'une fonction (cette notion sera présentée plus loin dans le cours).

Remarque 1.3 Quand on exécute l'une de ces deux commandes, R demande si l'on souhaite sauvegarder la session (c'est-à-dire enregistrer toutes les variables que l'on a créées dans un fichier sur le disque dur). Il faut toujours répondre non.

```
QUIT()
Quit()
```

Ces commandes ne fonctionnent pas car R est sensible à la casse c'est-à-dire R distingue les majusules des minuscules.

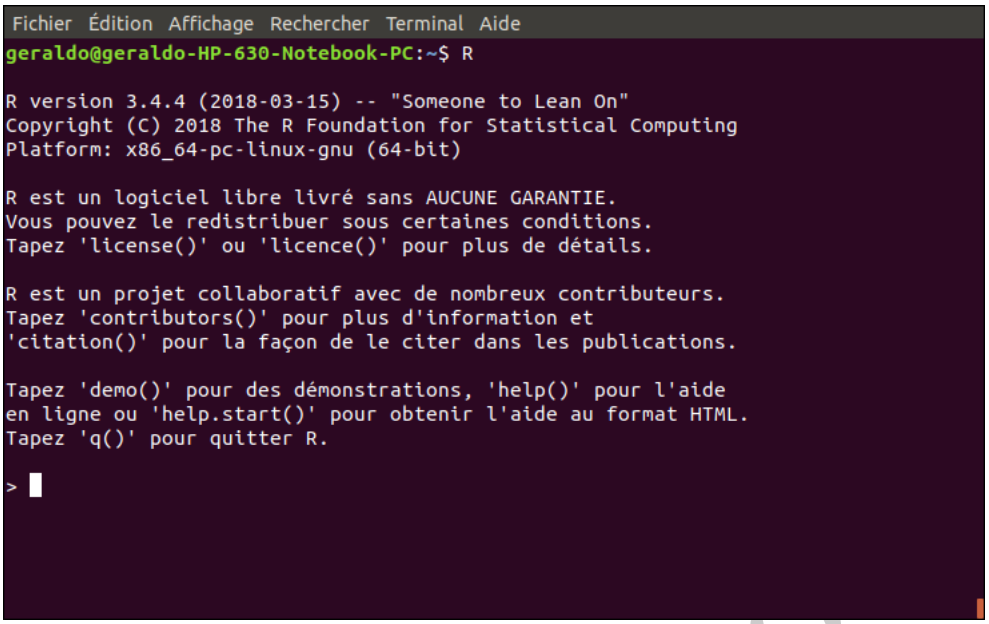


FIGURE 1.4 – Terminal Linux après lancement de R

1.4 COMMANDES ÉLÉMENTAIRES

1.4.1 Création de variables et affichage

```
x=2
x<-2
x <- 2
2 -> x
x=1+1
```

Commandes équivalentes permettant de créer (s'il n'existe pas encore) un objet `x` et de lui affecter la valeur 2.

```
employe.salaire2 = 100
```

Les noms de variables peuvent contenir des lettres, des chiffres, les caractères "_" et "."

Remarque 1.4 Même si les noms de variables sont très flexibles dans R, il y a quelques règles importantes à respecter.

1) Les noms de variables ne peuvent pas commencer par un chiffre ou un caractère spécial.
2) Comme dit plus haut, R est sensible à la casse donc les noms des variables sont aussi sensibles à la casse (les variables `x` et `X` sont deux variables différentes).
3) Quelques noms (comme `c`, `q`, `t`, `C`, `D`, `F`, `I`, `pi`, `T`, etc...) sont déjà utilisés par R pour désigner des objets internes et doivent être évités.

`x` `print(x)`	Affiche la valeur de l'objet `x`.
`print("Bonjour")`	Affichage de texte.

1.4.2 Séparation de commandes

Pour écrire plusieurs commandes, on peut :

- soit les aligner sur une même ligne en les séparant obligatoirement par un point-virgule
- soit passer à une nouvelle ligne pour chaque commande (dans ce cas le point-virgule n'est pas obligatoire à la fin d'une commande).

`x=2; y=3; z=-1`	Commandes sur la même ligne séparées par un ";"

`x=2`
`y=3`
`z=-1`
	Commandes séparées par un retour à la ligne.

1.4.3 Espace de travail

`objects()` `ls()`	Deux commandes équivalentes permettant d'obtenir la liste des objets présents en mémoire.
`rm(a,b)`	Supprime les objets `a` et `b` de la mémoire.
`rm(list=ls())`	Supprime tous les objets en mémoire.
`history()`	Historique des commandes entrées dans le terminal dans la session courante.
`getwd()`	Afficher le répertoire courant (le répertoire dans lequel R écrit ou lit les fichiers).
`setwd("C:/Users/Toto/")`	Commande permettant de définir `"C:/Users/Toto/"` comme répertoire courant. Remarquer l'utilisation de "/" et non de "\". Ainsi, si le nom du répertoire contient des "\", il faut les changer en "/".

1.4.4 Documentation de R

`help(solve)` `?solve`	Deux commandes équivalentes pour afficher l'aide HTML sur la fonction `solve`.
`help.search("bin")` `??bin`	Affiche la liste de toutes les pages de la documentation de R contenant le mot "bin".

1.4.5 Commentaires

`x=10 # Création de x`	Le symbole `#` est utilisé pour faire les commentaires **sur une seule ligne**. Sur une ligne donnée, R n'exécute pas le texte ou les commandes rencontrées après le symbole `#`.

1.5 Méthodes d'exécution des commandes

1.5.1 Première méthode

On peut entrer les commandes les unes après les autres dans la console de R et appuyer sur la touche **Enter** ↩ pour les exécuter (voir Figure 1.5).

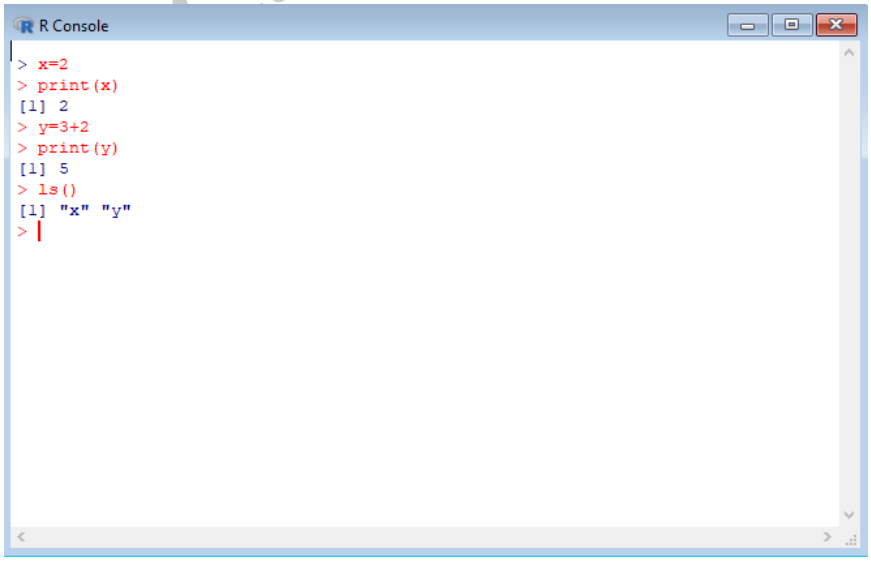


FIGURE 1.5 – Exemple d'exécution de commandes dans la console de R.

1.5.2 Deuxième méthode

On peut aussi créer un fichier de commandes (fichier d'extension `.R`) en utilisant l'éditeur de R (ou un autre éditeur).

Dans l'interface RGUI, l'éditeur de R est disponible en cliquant sur **`Fichier`** puis sur **`Nouveau script`** (voir figures 1.6 et 1.7).

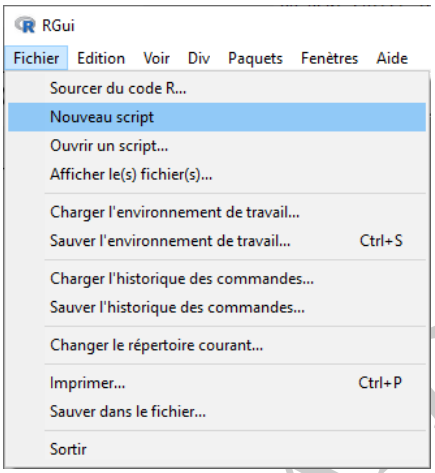


FIGURE 1.6 – Création d'un nouveau script dans RGUI.

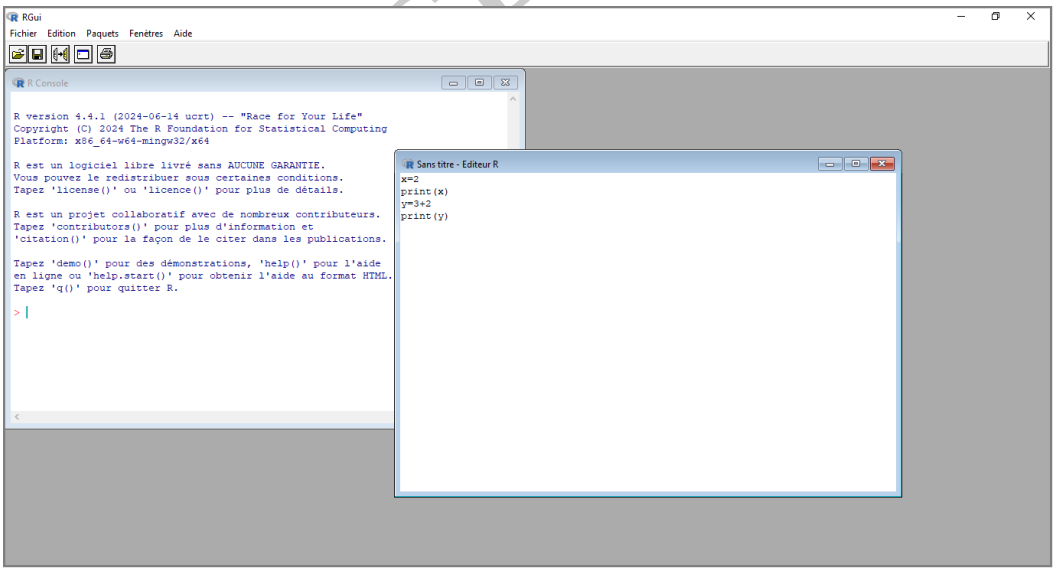


FIGURE 1.7 – Editeur de scripts de RGUI.

Après saisie du code dans l'éditeur et enregistrement sur le disque dur sous la forme d'un fichier d'extension `.R`, on peut l'exécuter dans la console à l'aide de la commande **`source`** comme suit :

`source("somme.R")`	Commande permettant d'exécuter l'ensemble des commandes contenues dans le fichier `somme.R`

Par défaut, R cherchera le fichier `somme.R` dans le répertoire courant (résultat de la commande `getwd()`). Si le fichier à exécuter se trouve dans un autre répertoire, il faudra donc au préalable utiliser la commande `setwd()` pour indiquer à R le répertoire dans lequel se trouve ledit fichier.

1.6 L'environnement de développement intégré RStudio

Comme dit plus tôt, sous Windows, R est installé avec l'interface RGUI. Mais il existe un logiciel gratuit du nom de **RStudio** offrant une interface graphique beaucoup plus élaborée facilitant le développement (ou la production) de codes dans le langage R. Dans le langage informatique, un tel logiciel est appelé **environnement de développement intégré**.

Le logiciel RStudio est téléchargeable à l'adresse :

https://posit.co/download/rstudio-desktop/

L'interface de RStudio se présente comme suit :

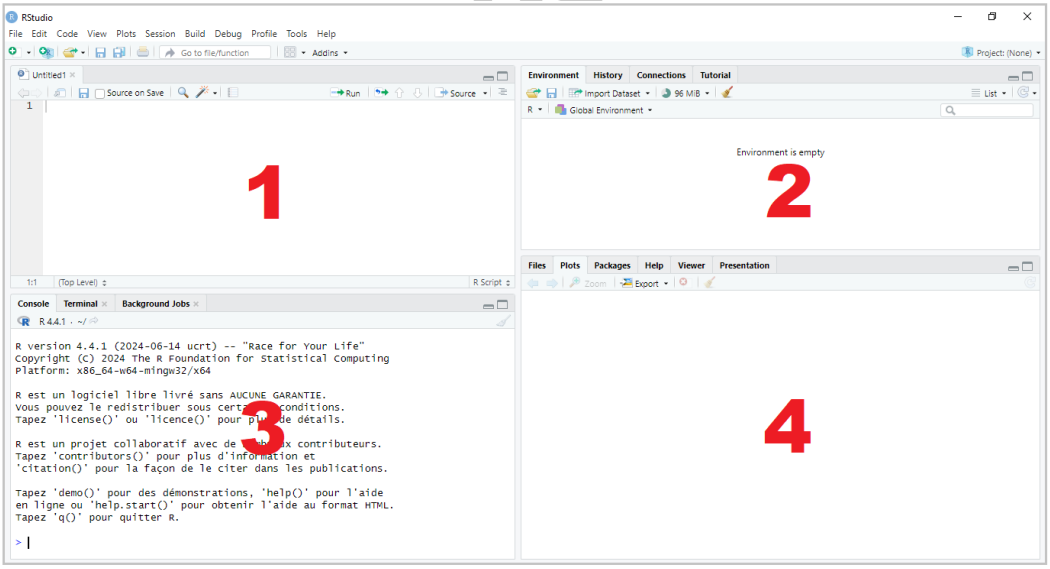


FIGURE 1.8 – Aperçu de RStudio.

Elle est composée de quatre (4) sous-fenêtres décrites ci-après :

- **Sous-fenêtre 1** : éditeur pour saisir les codes ;
- **Sous-fenêtre 2** : affichage des variables créées et de l'historique des commandes ;
- **Sous-fenêtre 3** : console ;
- **Sous-fenêtre 4** : affichage des graphiques et de la documentation.

Remarque 1.5 Au démarrage de RStudio, ce dernier vérifie que R est installé sur l'ordinateur. **Le logiciel RStudio ne remplace donc pas R mais vient en complément afin de faciliter le développement de codes R.**

1.7 BIBLIOTHÈQUES OU PACKAGES

Une bibliothèque (ou package) est un ensemble de fonctions et de données permettant de faire des tâches bien précises. Par exemple, la bibliothèque `stats` contient un ensemble de fonctionnalités permettant de faire des statistiques.

Il existe deux types de bibliothèques :

- les bibliothèques de base qui sont déjà présentes dans l'installation de base de R : seules certaines d'entre elles sont chargées automatiquement au démarrage de R (exemple des bibliothèques `base` et `stats`).
- d'autres bibliothèques écrites par d'autres utilisateurs de R disponibles sur le site du CRAN (Comprehensive R Archive Network) [1] :
  https://cran.r-project.org/web/packages/index.html.
  En fonction du besoin de l'utilisateur, elles peuvent être téléchargées puis installées grâce à certaines commandes présentées ci-après.

`library()`	Liste toutes les bibliothèques (packages) disponibles localement sur l'ordinateur où R est exécuté.
`library(stats)`	Chargement de la bibliothèque `stats`. Cette commande est indispensable pour utiliser les fonctions et données disponibles dans ladite bibliothèque.
`help(package=stats)`	Documentation du package `stats`.
`search()`	Liste des bibliothèques chargées.
`install.packages("nom")`	Télécharge la dernière version de la bibliothèque `nom` et l'installe.
`old.packages()`	Indique les packages installés localement sur l'ordinateur qui ne sont pas à jour c'est-à-dire les packages dont la dernière version publiée sur le site internet du CRAN est plus récente que celle installée localement.

1. En novembre 2024, le site du CRAN compte un peu plus de 21 700 packages.

I. C. GERALDO

`update.packages()`	Propose de télécharger et installer les versions mises à jour des packages installés.
`new.packages()`	Affiche la liste des packages disponibles sur le site internet du CRAN et qui n'ont pas encore été installés localement sur l'ordinateur de l'utilisateur.
`citation("nom")`	Indique comment citer la bibliothèque `nom` dans une publication (article, livre, mémoire, etc...).

A l'exécution de la commande d'installation d'un package, R peut demander de spécifier un miroir. Les miroirs sont des serveurs situés à plusieurs endroits dans le monde et disposant des mêmes fichiers que ceux téléchargeables sur le site du CRAN.

Remarque 1.6 Dans ce cours, nous n'aurons besoin de télécharger aucun package. La version basique de R contient toutes les fonctionnalités dont nous avons besoin.

I. C. GERALDO

CHAPITRE 2

CLASSES D'OBJETS ET FONCTIONS USUELLES

Sommaire

2.1 Introduction . . . 19
2.2 Vecteurs de nombres réels (classe `numeric`) et d'entiers relatifs (classe `integer`) . . . 19
2.2.1 Valeurs particulières . . . 19
2.2.2 Création . . . 19
2.2.3 Opérations et fonctions . . . 21
2.2.4 Composantes . . . 24
2.2.5 Opérations entières . . . 24
2.2.6 Nommer les composantes d'un vecteur . . . 24
2.3 Vecteurs de nombres complexes (classe `complex`) . . . 25
2.4 Matrices (classe `matrix`) . . . 26
2.4.1 Création et accès aux composantes . . . 26
2.4.2 Opérations et fonctions matricielles . . . 27
2.4.3 Nommer les lignes et colonnes d'une matrice . . . 29
2.5 Vecteurs de chaînes de caractères (classe `character`) et facteurs (classe `factor`) . . . 30
2.5.1 Vecteurs de chaînes de caractères (classe `character`) . . . 30
2.5.2 Facteurs (classe `factor`) . . . 32
2.6 Listes (classe `list`) . . . 33
2.7 Tableaux de données (classe `data.frame`) . . . 33
2.7.1 Création par lignes de commandes . . . 33
2.7.2 Jeux de données internes . . . 34
2.7.3 Importation de tableaux de données . . . 35
2.7.4 Manipulation . . . 36
2.8 Vecteurs logiques (classe `logical`) . . . 39
2.8.1 Création par saisie . . . 39
2.8.2 Opérations sur les vecteurs logiques . . . 39
2.8.3 Opérateurs de comparaison . . . 40
2.8.4 Opérateurs (connecteurs) logiques . . . 40
2.8.5 Fonctions logiques usuelles . . . 41
2.8.6 Sélection d'éléments dans un vecteur . . . 42
2.8.7 Sélection d'éléments dans un tableau (matrice ou tableau de données) 42
2.9 Exercices . . . 44

I.C. GERALDO

2.1 INTRODUCTION

Tous les objets manipulés dans R (variables et fonctions) ont une classe et les manipulations possibles sur un objet donné dépendent de sa classe. Les classes d'objets usuelles sont :

- `numeric`
- `integer`
- `complex`
- `matrix`
- `character`
- `factor`
- `list`
- `data.frame`
- `logical`

`x=2; class(x)`	Affichage de la classe de l'objet `x`, ici `"numeric"`.

2.2 VECTEURS DE NOMBRES RÉELS (CLASSE `numeric`) ET D'ENTIERS RELATIFS (CLASSE `integer`)

La manipulation des deux classes est pratiquement la même.

2.2.1 Valeurs particulières

`pi`	Variable interne contenant la valeur de π.
`2e-3; 2E-3`	Exemples de format scientifique pour désigner $2 \times 10^{-3} = 0.002$.
`Inf ; -Inf`	Respectivement $+\infty$ et $-\infty$.
`1/0 ; -1/0`	Respectivement `Inf` et `-Inf`.
`NA`	Valeur à saisir pour indiquer une valeur manquante (Not Available).
`0/0 ; Inf-Inf; Inf/Inf`	`NaN` (Not a Number).

2.2.2 Création

Il existe plusieurs méthodes pour créer un vecteur de nombres réels ou entiers relatifs. Dans la pratique, il revient à l'utilisateur de choisir la méthode la mieux adaptée à ce qu'il souhaite faire.

`x = numeric(5)`	Création d'un vecteur de classe `numeric` contenant 5 fois la valeur zéro (0).
`x = integer(5)`	Création d'un vecteur de classe `integer` contenant 5 fois la valeur zéro (0).

`x = c(1,3,4.7,-5)`	Création du vecteur $(1, 3, 4.7, -5)$ affecté à `x`. La lettre `c` peut être comprise comme *collect* ou *combine*. Le point "." est le symbole décimal.
`print(x)`	Affichage de `x`
`0:5`	Vecteur $(0, 1, 2, 3, 4, 5)$.
`seq(from=1,to=10,by=2)`	Création d'un vecteur dont les composantes forment une suite arithmétique de premier terme 1 (`from=1`), dont la raison (encore appelée incrément ou pas) vaut 2 (`by=2`). L'argument `to=10` indique que la composante maximale ne doit pas dépasser 10. Ici le résultat est un vecteur à 5 composantes : 1, 3, 5, 7 et 9.
`seq(from=1,to=10,length.out=5)`	Création d'un vecteur de 5 composantes formant une suite arithmétique de premier terme 1 (`from=1`) et de dernier terme 10, dont la raison vaut $$\text{raison} = \frac{10-1}{5-1} = 2.25$$ Ici, l'argument `to=10` indique que la dernière composante doit être exactement 10. Ici le résultat est un vecteur à 5 composantes : 1, 3.25, 5.5, 7.75 et 10.
`x=c(1,2);` `c(x,x,x); rep(x,times=3)`	Commandes équivalentes permettant de créer un nouveau vecteur formé en mettant `x` bout à bout 3 fois. Le résultat est $(1, 2, 1, 2, 1, 2)$.
`rep(x,times=c(2,3))`	Création d'un vecteur contenant 2 fois la première composante et 3 fois la seconde. Le résultat est $(1, 1, 2, 2, 2)$.
`rep(x,each=3)`	Création d'un vecteur dans lequel chaque composante de `x` est répétée 3 fois successivement. Le résultat est $(1, 1, 1, 2, 2, 2)$.

I. C. GE

`rep(x,length.out=5)`	Création d'un vecteur dans lequel les composantes de x sont répétées de façon alternée jusqu'à obtenir 5 composantes. Le résultat est $(1,2,1,2,1)$.

2.2.3 Opérations et fonctions

2.2.3.1 Opérations et fonctions usuelles

Soient x et y deux vecteurs créés à l'aide du code ci-après :

```
x=c(3,0,7,2.5)
y=c(2,3,-4,1)
```

`z=x+y`
`z=x*y`
`z=x/y`
	Addition, multiplication et division composante par composante pour deux vecteurs de même nombre de composantes.

`2*x`	Multiplication de toutes les composantes de x par 2.
`x+2`	Ajoute 2 à toutes les composantes de x.
`x^2; x**2`	Commandes équivalents permettant d'élever chaque composante de x au carré.

`abs(x); x^(1/3); sqrt(x);`
`exp(x); log(x); log2(x);`
`log10(x); sin(x); cos(x);`
`tan(x); sinh(x); cosh(x);`
`tanh(x); asin(x); acos(x);`
`atan(x); ...`
	Ces fonctions usuelles s'appliquent à toutes les composantes de x c'est-à-dire elles retournent un vecteur de même taille que x dont les composantes sont les images respectives de celles de x.

`length(x)`	Nombre de composantes de x.
`max(x) ; min(x)`	Maximum et minimum des composantes de x.
`range(x)`	Vecteur composé du minimum et du maximum des composantes de x.
`sum(x) ; mean(x)`	Somme des composantes, moyenne.
`round(x); round(x,2)`	Arrondi à l'entier le plus proche ; arrondi à la seconde décimale.
`ceiling(x)`	Plus petit entier supérieur.

`floor(x)`	Fonction partie entière (plus grand entier inférieur).
`trunc(x)`	Troncature des décimales.
`factorial(x)`	Calcule les factorielles des éléments du vecteur `x`.
`choose(5,3)`	Valeur de C_5^3.
`rev(x)`	Renverse le vecteur `x`. Résultat : $(2.5, 7, 0, 3)$.
`sort(x)`	Tri par ordre croissant.
`sort(x,decreasing=TRUE)`	Tri par ordre décroissant.
`order(x)`	Renvoie l'ordre dans lequel il faut sélectionner les composantes de `x` pour avoir le tri par ordre croissant. Ici, on obtient $2, \quad 4, \quad 1, \quad 3,$ ce qui signifie que pour trier par ordre croissant, il faut prendre le 2ème élément de `x` (c'est-à-dire 0), puis le 4ème (2.5), puis le 1er (3) et enfin le 3ème (7).
`order(x,decreasing=TRUE)`	Renvoie l'ordre dans lequel il faut sélectionner les composantes de `x` pour avoir le tri par ordre décroissant.
`which.min(x)`	Position du minimum. Retourne 2 (le deuxième élément de `x` est le minimum). Cette fonction ne prend pas en compte les doublons c'est-à-dire au cas où le minimum est présent plusieurs fois, elle retourne la première position où se trouve ce dernier.
`which.max(x)`	Position du maximum. Retourne 3 (le troisième élément de `x` est le maximum). Tout comme `which.min`, cette fonction ne prend pas en compte les doublons c'est-à-dire au cas où le maximum est présent plusieurs fois, elle retourne la première position où se trouve ce dernier.

`rank(x,ties.method="min")`	Renvoie un vecteur de même longueur que `x` en indiquant le rang de chaque élément dans le classement par ordre croissant des éléments de `x`. L'avantage de cette fonction est qu'elle prend en compte les ex æquo. Pour obtenir le rang dans l'ordre décroissant, il suffit de remplacer `x` par `-x`.

`x=c(3,0,7,2.5)`
`y=c(2,3,-4,1)`
`pmax(x,y)`
	La fonction `pmax` (parallel maxima) retourne le maximum pour chaque position. Le premier élément du résultat est le maximum des premiers éléments de `x` et `y` (c'est-à-dire `max(x[1],y[1])`), le deuxième élément du résultat est le maximum des deuxièmes éléments et ainsi de suite.
	Ici, le résultat est : 3, 3, 7, 2.5.

`pmin(x,y)`	La fonction `pmin` (parallel minima) retourne le minimum pour chaque position. Ici, le résultat est : 2, 0, -4, 1.

`intersect(x,y)`
`union(x,y)`
`setdiff(x,y)`
	Etant donnés deux vecteurs `x` et `y`, ces commandes permettant de récupérer respectivement (sous forme de vecteurs) :
	• les éléments communs à `x` et `y`,
	• la réunion des éléments de `x` et `y`,
	• les éléments de `x` qui ne sont pas éléments de `y`.

2.2.3.2 De la nécessité de l'utilisation de parenthèses

`1:4-1 ; 1:(4-1)`	Respectivement $(0,1,2,3)$ et $(1,2,3)$. Dans le premier cas, R exécute la commande `1:4` pour former le vecteur $(1,2,3,4)$ puis retranche 1 à toutes les composantes. Le second cas est équivalent à `1:3`.
`2^2^3 ; (2^2)^3`	Respectivement $2^8 = 256$ et $4^3 = 64$.

2.2.4 Composantes

`x[1]`	La 1ère composante de `x`.
`x[1:4]`	Les quatre premières composantes de `x`.
`x[c(2,3)]`	Les 2ème et 3ème composantes de `x`.
`x[-c(1,3)]`	Toutes les composantes de `x` sauf la première et la troisième.
`head(x,2)`	Extraction des 2 premiers éléments.
`tail(x,2)`	Extraction des 2 derniers éléments.
`x[2]=-3`	La 2ème composante de `x` est remplacée par 2.

2.2.5 Opérations entières

`x%%2`	Renvoie pour chaque élément du vecteur numérique `x`, le reste de la division euclidienne dudit élément par 2.
`x%/%2`	Renvoie pour chaque élément du vecteur numérique `x`, le quotient de la division euclidienne dudit élément par 2.

2.2.6 Nommer les composantes d'un vecteur

Exemple 2.1 Lors de l'année scolaire 2020-2021, l'effectif des élèves dans le second cycle au Togo est réparti par région comme suit (source : https://togo.dataforall.org/) :

DAGL	Maritime	Plateaux	Centrale	Kara	Savanes
58 560	34 386	34 385	17 070	24 703	18 048

DAGL = District Autonome du Grand Lomé

On peut saisir ces données à l'aide d'un vecteur numérique avec des composantes (ou colonnes) nommées. Pour ce faire, il existe deux possibilités.

- On peut utiliser la commande `c` comme suit.

Console R

```
> x=c(58560,34386,34385,17070,24703,18048)
> print(x)
[1] 58560 34386 34385 17070 24703 18048
> y=c(DAGL=58560,Maritime=34386,Plateaux=34385,Centrale=17070,Kara=24703,
+ Savanes=18048)
> print(y)
```

```
    DAGL Maritime Plateaux Centrale     Kara  Savanes
   58560    34386    34385    17070    24703    18048
> z=c("DAGL"=58560,"Maritime"=34386,"Plateaux"=34385,"Centrale"=17070,"Kara"=24703,
+ "Savanes"=18048)
> print(z)
    DAGL Maritime Plateaux Centrale     Kara  Savanes
   58560    34386    34385    17070    24703    18048
```

- On peut aussi utiliser la commande `names(x)` permettant d'afficher les noms des colonnes de `x` comme suit.

Console R

```
> x=c(58560,34386,34385,17070,24703,18048)
> print(x)
[1] 58560 34386 34385 17070 24703 18048
> names(x)=c("DAGL","Maritime","Plateaux","Centrale","Kara","Savanes")
> print(x)
    DAGL Maritime Plateaux Centrale     Kara  Savanes
   58560    34386    34385    17070    24703    18048
```

Ici aussi, on voit bien la différence à l'affichage.

On peut aussi utiliser les noms des colonnes pour sélectionner les éléments du vecteur comme suit :

`x[2] ; x["Maritime"]`	Commandes équivalentes pour sélectionner le 2ème élément du vecteur `x`.

2.3 VECTEURS DE NOMBRES COMPLEXES (CLASSE `complex`)

`sqrt(-1)`	Affiche `NaN` (Not a Number).
`sqrt(-1+0i)`	Affiche `0+1i`.
`z=0+0i`	Le zéro complexe ($0 \in \mathbb{C}$). Remarquez que l'on n'a pas fait `0+0*i` mais bien `0+0i`.
`z=c(-1+2i,-1+0i)`	Création d'un vecteur de nombres complexes.
`z=complex(5)`	Création d'un vecteur formé de 5 fois le nombre complexe nul.
`Re(z); Im(z)`	Retournent respectivement les parties réelles et imaginaires des éléments de `z`.
`Conj(z)`	Les conjugués respectifs des composantes de `z`.

I. C. GERALDO

`Mod(z); Arg(z)`	Retournent respectivement les modules et les arguments des composantes de `z`.

Avec les vecteurs de nombres complexes, on peut faire les opérations qui ont du sens telles que l'addition, la soustraction, la multiplication, la division et le calcul de puissances entières.

2.4 MATRICES (CLASSE `matrix`)

2.4.1 Création et accès aux composantes

`A=matrix(0,nrow=2,ncol=3)` `A=matrix(0,2,3)`	Commandes équivalentes permettant de créer une matrice A à 2 lignes et 3 colonnes composée de 0.
`A=matrix(1:6,nrow=2,ncol=3)` `A=matrix(1:6,2,3)`	Commandes équivalentes permettant de créer une matrice à 2 lignes et 3 colonnes dont les composantes sont les nombres de 1 à 6. Par défaut, la matrice est remplie colonne par colonne c'est-à-dire $$A = \begin{pmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{pmatrix}.$$

`A=matrix(c(-1,-2,3,0,4,7),`
`nrow=2, ncol=3)`
 	
`A=matrix(c(-1,3,4,-2,0,7),`
`nrow=2, ncol=3,`
`byrow=TRUE)`
	Commandes équivalentes permettant de créer la matrice $$A = \begin{pmatrix} -1 & 3 & 4 \\ -2 & 0 & 7 \end{pmatrix}.$$ L'argument `byrow=TRUE` permet de spécifier que le remplissage doit se faire suivant les lignes.

`A[1,3]`	Elément situé à l'intersection de la première ligne et de la $3^{\text{ème}}$ colonne.
`A[1,]`	Les éléments de la première ligne.
`A[,2]`	Les éléments la $2^{\text{ème}}$ colonne.
`A[1:2,2:3]`	Sous-matrice composée des deux premières lignes et des $2^{\text{ème}}$ et $3^{\text{ème}}$ colonnes.

2.4.2 Opérations et fonctions matricielles

`nrow(A) ; ncol(A)`	Nombres de lignes et de colonnes de la matrice `A`.
`dim(A)`	Vecteur de deux composantes contenant respectivement le nombre de lignes et le nombre de colonnes de `A`.
`row(A)`	Commande retournant une matrice de mêmes dimensions que A dans laquelle chaque élément est remplacé par son numéro de ligne.
`col(A)`	Commande retournant une matrice de mêmes dimensions que A dans laquelle chaque élément est remplacé par son numéro de colonne.

Console R

```
> A=matrix(c(2,-1,4,0,7,-2),2,3)
> A
     [,1] [,2] [,3]
[1,]    2    4    7
[2,]   -1    0   -2
> row(A)
     [,1] [,2] [,3]
[1,]    1    1    1
[2,]    2    2    2
> col(A)
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    1    2    3
```

I. C. GERALDO

`A+B ; A*B ; A^2`	Opérations termes à termes : `A` et `B` doivent être de même dimension.
`A%*%B`	Produit matriciel (nécessite que le nombre de colonnes de `A` soit égal au nombre de lignes de `B`).
`t(A)`	Transposée.
`diag(v)`	Si `v` est un vecteur, cette commande donne une matrice diagonale avec les éléments de `v` sur la diagonale.

`diag(A)`	Si `A` est une matrice, cette commande permet d'extraire la diagonale de `A` sous forme d'un vecteur.
`diag(k)`	Si `k` est un nombre entier naturel, cette commande permet d'obtenir la matrice identité d'ordre `k`.
`det(A)`	Déterminant de la matrice carrée `A`.
`solve(A)`	Inverse de `A`.
`solve(A,B)`	Résolution de l'équation `AX=B` où `A` est une matrice carrée d'ordre n inversible et `B` est un vecteur de taille n ou une matrice à n lignes.
`eigen(A)$values` `eigen(A)$vectors`	Valeurs propres de `A`. Vecteurs propres de `A`.
`apply(A,dim,f)`	Applique la fonction `f` à chaque ligne de `A` si `dim=1` et chaque colonne si `dim=2`.
`apply(A,1,mean); rowMeans(A)`	Commandes équivalentes pour calculer la moyenne des éléments de `A` par ligne.
`apply(A,2,mean); colMeans(A)`	Commandes équivalentes pour calculer la moyenne des éléments de `A` par colonne.

`apply(A,1,sum)`
`rowSums(A)`
`marginSums(A,1)`
`margin.table(A,1)`
	Commandes équivalentes pour calculer la somme des éléments de `A` par ligne.

`apply(A,2,sum)`
`colSums(A)`
`marginSums(A,2)`
`margin.table(A,2)`
	Commandes équivalentes pour calculer la somme des éléments de `A` par colonne.

`addmargins(A)`	Rajoute à la matrice `A` des marges contenant respectivement les sommes par lignes et par colonnes.

`addmargins(A,FUN=mean)`	Rajoute à la matrice `A` des marges contenant respectivement les moyennes par lignes et par colonnes. On comprend ainsi que lorsque l'argument `FUN` n'est pas précisé, c'est la somme qui est appliquée par défaut.
`addmargins(A,1)`	Rajoute en bas de la matrice `A`, une ligne contenant les sommes par colonnes.
`addmargins(A,2)`	Rajoute à droite de la matrice `A`, une colonne contenant les sommes par lignes.
`cbind(A,B,C)`	Concaténation horizontale des matrices `A`, `B` et `C`.
`rbind(A,B,C)`	Concaténation verticale des matrices `A`, `B` et `C`.
`outer(x,y,"*"); outer(x,y,"+")`	Produit cartésien des vecteurs numériques `x` et `y` et application des opérateurs * et +.

2.4.3 Nommer les lignes et colonnes d'une matrice

Les commandes `rownames` et `colnames` permettent d'attribuer des noms aux lignes et aux colonnes d'une matrice.

Exemple 2.2

Console R

```
> notes=matrix(c(12,15,7,13,11,20),2,3,byrow=TRUE)
> print(notes)
     [,1] [,2] [,3]
[1,]   12   15    7
[2,]   13   11   20
> rownames(notes)=c("Kodjo","Afi")
> colnames(notes)=c("MTH","PHY","SVT")
> print(notes)
      MTH PHY SVT
Kodjo  12  15   7
Afi    13  11  20
```

Ici aussi, on voit bien la différence à l'affichage.

On peut aussi utiliser les noms des lignes et des colonnes pour sélectionner les éléments d'une matrice comme suit :

`notes[1,2] ;` `notes["Kodjo","PHY"]`	Commandes équivalentes permettant d'extraire la note de Kodjo en PHY.
`notes[,1] ; notes[,"MTH"]`	Commandes équivalentes permettant d'extraire toutes les notes de MTH (première colonne).
`notes[2,] ; notes["Afi",]`	Commandes équivalentes permettant d'extraire les notes de Afi (deuxième ligne).

2.5 VECTEURS DE CHAÎNES DE CARACTÈRES (CLASSE `character`) ET FACTEURS (CLASSE `factor`)

Ces deux classes permettent de saisir des données qualitatives. La principale différence est que les facteurs (classe `factor`) sont beaucoup plus adaptés aux variables qualitatives ayant des modalités alors que les chaînes de caractères simples (classe `character`) sont souvent utilisées pour des données qualitatives n'ayant pas a priori un caractère statistique (par exemple, le nom et les prénoms).

2.5.1 Vecteurs de chaînes de caractères (classe `character`)

2.5.1.1 Création et manipulation

`nom = c("Kodjo","Afi")`	Création d'un vecteur `nom` vecteur composé de deux chaînes de caractères. Ne pas oublier l'emploi des "".
`character(5)`	Création d'un vecteur composé de 5 chaines de caractères vides.
`paste("bon","jour")`	La commande `paste` permet de coller (concaténer) autant de chaînes de caractères que l'on veut. Ici, le résultat est la chaîne de caractères "bon jour". Par défaut, un espace sépare les chaînes concaténées.
`paste("bon","jour",sep="")`	Résultat : "bonjour". Ici, l'on a précisé le séparateur (`sep`) qui est une chaîne de caractères vide.
`paste(1:3,c("A","B","C"),sep="")`	On peut aussi concaténer des vecteurs. Ici le résultat est le vecteur de chaînes de caractères : `"1A" "2B" "3C"`

I. C. GERALDO

`as.character(10)`	Le nombre 10 devient la chaîne de caractères "10".
`as.numeric("10")`	La chaîne de caractères "10" devient le nombre 10.
`nchar("Bonjour")`	Nombre de caractères de la chaîne : ici 7.
`toupper("bon"); tolower("BON")`	Conversion en majuscules / minuscules.
`substr("ABCDE",2,4)` `substring("ABCDE",2,4)`	Commandes équivalentes permettant l'extraction des caractères de la 2ème à la 4ème position sous la forme d'une chaîne de caractères. Le résultat est `"BCD"`.
`substring("ABCDE",1:2,3:4)` `substring("ABCDE",c(1,2),c(3,4))`	Commandes équivalentes permettant l'extraction des caractères de la 1ère à la 3ème position puis de la 2ème à la 4ème position sous la forme de chaînes de caractères. Le résultat est un vecteur de deux chaînes de caractères : `"ABC"` et `"BCD"`.
`chartr("12345", "ABCDE", x)`	Remplace respectivement 1, 2, 3, 4 et 5 par A, B, C, D et E dans le vecteur de chaînes de caractères `x`.
`unique(x)`	Retourne un objet de même classe que `x` en enlevant les doublons.

`unlist(strsplit(mot,NULL))`
`substring(mot, 1:length(mot),`
`1:length(mot))`
	Commandes équivalentes retournant un vecteur de chaînes de caractères contenant la chaîne de caractères `mot` décomposée par lettres.

`help(character)`	Ouverture de la documentation de R sur les chaînes de caractères.

2.5.1.2 Vecteurs de chaînes de caractères internes à R

Le logiciel R contient des vecteurs de chaînes de caractères prédéfinis sous forme de constantes internes. Parmi eux, on peut citer :

`LETTERS`	$\longrightarrow$	Les 26 lettres de l'alphabet en majuscules.
`letters`	$\longrightarrow$	Les 26 lettres de l'alphabet en minuscules.
`month.name`	$\longrightarrow$	Les 12 mois de l'année en anglais .
`month.abb`	$\longrightarrow$	Les abréviations de 3 lettres de `month.name`.

2.5.2 Facteurs (classe `factor`)

Cette classe d'objets permet de saisir des données concernant une variable statistique qualitative ayant plusieurs modalités.

Pour saisir des données d'une variable statistique qualitative ayant plusieurs modalités, on utilise la fonction **`factor`** comme dans l'exemple ci-après.

Exemple 2.3 Considérons les mentions obtenues par 10 élèves admis au BAC2 :

B TB P P AB P B P B TB

où les mentions sont codées comme suit : P = Passable, AB = Assez Bien, B = Bien et TB = Très Bien. On peut saisir les données et les afficher avec le code suivant :

Console R

```
> mention = factor(c("B","TB","P","P","AB","P","B","P","B","TB"))
> print(mention)
 [1] B TB P P AB P B P B TB
Levels: AB B P TB
```

La deuxième ligne de l'affichage donne les modalités (levels).

`levels(mention)`	Liste des modalités.
`summary(mention)` `table(mention)`	Deux commandes équivalentes permettant d'obtenir le tableau de contingence du vecteur **`mention`** c'est-à-dire un tableau donnant les modalités et le nombre de fois que chacune apparaît.

Console R

```
> print(table(mention))
mention
AB  B  P TB
 1  3  4  2
```

Dans ce cas particulier, on remarque que R a trié les modalités par ordre alphabétique (`AB`, `B`, `P` et `TB`). L'on peut indiquer à R de considérer les modalités dans l'ordre usuel des mentions (`P`, `AB`, `B` et `TB`) comme suit.

Console R

```
> mention = factor(mention, levels=c("P","AB","B","TB"))
> print(table(mention))
mention
 P AB  B TB
 4  1  3  2
```

I. C. GERALDO

Il est possible d'associer les codes des mentions à leurs vrais intitulés comme suit.

Console R

```
> mention=factor(mention, levels=c("P","AB","B","TB"), labels=c("Passable","Assez Bien",
+ "Bien","Très Bien"))
> print(table(mention))
mention
  Passable Assez Bien       Bien  Très Bien
         4          1          3          2
```

Remarque 2.1 L'ordre des intitulés dans `labels` doit correspondre à celui des composantes de `levels`.

2.6 Listes (classe `list`)

Une liste est une collection d'objets dont les classes peuvent être différentes. Les listes présentent un intérêt particulier lors de la création et de la manipulation des fonctions que nous verrons plus tard.

`etud=list(nom="Kodjo", age=21, genre="M")`	Création d'une liste à trois composantes de différentes classes : les composantes `nom` et `genre` sont de classe `character` alors que `age` est de classe `numeric`.
`names(etud)`	Noms des composantes de la liste `etud`.
`etud$age ; etud[[2]] ; etud[["age"]]`	Commandes équivalentes pour obtenir la seconde composante `age`.

2.7 Tableaux de données (classe `data.frame`)

Les tableaux de données (ou structures de données hétérogènes) sont des tableaux dont les lignes sont des listes de mêmes composantes et dont les colonnes peuvent être de différents types (numérique, logique, chaînes de caractères). Leur manipulation (repérage des lignes et des colonnes) est très semblable à celle des matrices. Ce type de structure sert à stocker des informations (en colonnes) relatives aux individus (en lignes).

2.7.1 Création par lignes de commandes

Elle se fait à l'aide de la fonction **`data.frame`**.

Exemple 2.4 Considérons le tableau suivant donnant les notes de sept (7) étudiants à un examen de Statistique :

Nom	Note	Sexe	Parcours
Afi	17	F	MTH
Koffi	18	M	CHM
Kodjo	9	M	PHY
Adjo	11	F	CHM
Dodji	16	M	MTH
Abla	12	F	PHY
Abalo	20	M	MTH

CHM : Chimie, MTH : Maths, PHY : Physique.

Il s'agit d'un tableau de données de sept (7) lignes (individus) et quatre (4) colonnes (variables) avec des chaînes de caractères en 1ère colonne, du numérique dans la seconde et des facteurs dans les troisième et la quatrième colonnes.

- Pour saisir les données, on peut utiliser le code

```
x=data.frame(nom=c("Afi","Koffi","Kodjo","Adjo","Dodji","Abla","Abalo"),
             note=c(17,18,9,11,16,12,20),
             sexe=factor(c("F","M","M","F","M","F","M")),
             parcours=factor(c("MTH","CHM","PHY","CHM","MTH","PHY","MTH")))
```

ou le code

```
nom=c("Afi","Koffi","Kodjo","Adjo","Dodji","Abla","Abalo"),
note=c(17,18,9,11,16,12,20),
sexe=factor(c("F","M","M","F","M","F","M")),
parcours=factor(c("MTH","CHM","PHY","CHM","MTH","PHY","MTH"))
x=data.frame(nom,note,sexe,parcours)
```

- Pour l'affichage, on utilise le code

```
print(x)
```

qui donne comme résultat :

```
    nom note sexe parcours
1   Afi   17    F      MTH
2 Koffi   18    M      CHM
3 Kodjo    9    M      PHY
4  Adjo   11    F      CHM
5 Dodji   16    M      MTH
6  Abla   12    F      PHY
7 Abalo   20    M      MTH
```

2.7.2 Jeux de données internes

Ce sont des jeux de données installées avec R, accessibles à chaque lancement de R et qui n'ont pas besoin d'être créés par l'utilisateur.

I. C. GERALDO

`data()`	Commande permettant de lister tous les jeux de données internes disponibles.
`data(cars)`	Chargement du jeu de données interne nommé `cars`. La variable `cars` est maintenant disponible.
`help(cars)`	Affichage de la documentation sur le jeu de données interne nommé `cars`.

2.7.3 Importation de tableaux de données

On peut aussi importer des tableaux de données dans R depuis des fichiers d'extensions telles que `.txt`, `.csv`, `.xls` et `.xlsx`.

Un fichier de données doit être organisé de préférence, de la façon suivante :
- La première ligne peut contenir ou non les noms des composantes ou variables.
- Chaque ligne suivante correspond à un nouvel individu décrit par les variables précédentes. Le début de la ligne peut contenir ou non l'identifiant de cet individu.
- Les colonnes sont séparées par un espace ou par un autre séparateur.

Exemple 2.5 Voici un exemple de contenu de fichier à importer pour lequel les colonnes sont séparées par un espace :

example.txt

```
nom note sexe parcours
Afi 17 F MTH
Koffi 18 M CHM
Kodjo 9 M PHY
Adjo 11 F CHM
Dodji 16 M MTH
Abla 12 F PHY
Abalo 20 M MTH
```

Voici les mêmes données au format CSV avec séparateur point-virgule :

example.csv

```
nom;note;sexe;parcours
Afi;17;F;MTH
Koffi;18;M;CHM
Kodjo;9;M;PHY
Adjo;11;F;CHM
Dodji;16;M;MTH
Abla;12;F;PHY
Abalo;20;M;MTH
```

Suivant l'extension du fichier à importer, l'on pourra utiliser l'une des fonctions suivantes :

I. C. GERALDO

Fichiers `*.txt`	⟶	Fonction `read.table`
Fichiers `*.csv`	⟶	Fonctions `read.csv` ou `read.csv2`
Fichiers `*.xls` ou `*.xlsx`	⟶	Fonctions `read.xlsx` et `read.xlsx2` de la librairie `xlxs` (à télécharger, installer et charger).

L'importation de données n'est pas abordée en détails dans ce cours. Le lecteur intéressé pourra consulter la documentation ou les références bibliographiques du cours pour plus de détails sur les fonctions sus-citées.

2.7.4 Manipulation

`View(x)`	Affichage de `x` dans le tableur interne de R.
`names(x)`	Noms des composantes (colonnes) de `x`.
`str(x)`	Affichage de la structure de `x`.

Console R

```
> names(x)
[1] "nom"      "note"     "sexe"     "parcours"
> str(x)
'data.frame':       7 obs. of  4 variables:
 $ nom     : chr  "Afi" "Koffi" "Kodjo" "Adjo" ...
 $ note    : num  17 18 9 11 16 12 20
 $ sexe    : Factor w/ 2 levels "F","M": 1 2 2 1 2 1 2
 $ parcours: Factor w/ 3 levels "CHM","MTH","PHY": 2 1 3 1 2 3 2
```

`x$note ; x[,2] ; x[["note"]]; x[[2]]`	Commandes équivalentes permettant de récupérer la colonne `note` de `x`.
`attach(x)`	Commande permettant de désigner directement les colonnes de `x` par leurs noms sans les faire précéder par `x$`.
`detach(x)`	Commande permettant d'annuler l'effet de la commande `attach(x)`.

```
tapply(x$note,x$sexe,mean)
aggregate(note~sexe, data=x,
          FUN=mean)
aggregate(note~sexe, data=x,
          mean)
aggregate(x$note,list(x$sexe),mean)
```

Commandes équivalentes permettant de calculer la note moyenne par sexe.

```
tapply(x$note,
       list(x$sexe,x$parcours),
       mean)
aggregate(note~sexe+parcours,
          x, mean)
```

Commandes équivalentes permettant de calculer la note moyenne par sexe et par parcours.

Console R

```
> tapply(x$note,list(x$sexe,x$parcours),mean)
  CHM MTH PHY
F  11  17  12
M  18  18   9
> aggregate(note~sexe+parcours,data=x,mean)
  sexe parcours note
1    F      CHM   11
2    M      CHM   18
3    F      MTH   17
4    M      MTH   18
5    F      PHY   12
6    M      PHY    9
> aggregate(x$note,list(x$sexe,x$parcours),mean)
  Group.1 Group.2  x
1       F     CHM 11
2       M     CHM 18
3       F     MTH 17
4       M     MTH 18
5       F     PHY 12
6       M     PHY  9
```

I. C. GERALDO

```
y=transform(x,
       note2=pmin(note+1,20),
       lettre1=substr(nom,1,1))

y=within(x,
       {note2=pmin(note+1,20);
       lettre1=substr(nom,1,1)})
```

Commandes équivalentes permettant de créer, à partir de `x`, un nouveau tableau de données `y` contenant toutes les colonnes de `x` et deux nouvelles colonnes `note2` (obtenue en ajoutant 1 à `note`) et `lettre1` (obtenue par extraction de la première lettre de `nom`).

Pour `note2`, il faut remarquer la comparaison avec 20 qui est la note maximale et l'utilisation de `pmin` (*parallel minima*) au lieu de `min` car `pmin` agit composante par composante.

Console R

```
> y=transform(x,note2=pmin(note+1,20),lettre1=substr(nom,1,1))
> print(y)
    nom note sexe parcours note2 lettre1
1   Afi   17    F      MTH    18       A
2 Koffi   18    M      CHM    19       K
3 Kodjo    9    M      PHY    10       K
4  Adjo   11    F      CHM    12       A
5 Dodji   16    M      MTH    17       D
6  Abla   12    F      PHY    13       A
7 Abalo   20    M      MTH    20       A
```

`z=within(x,{rm(note)})`	Commande permettant de créer un tableau de données `z` contenant presque toutes les colonnes de `x` sauf la colonne `note`.
`x[order(x$parcours),]`	Tri des lignes de `x` par valeurs croissantes de la colonne `parcours`.
`x[order(x$parcours,x$nom),]`	Tri des lignes de `x` par valeurs croissantes de la colonne `parcours` puis suivant les valeurs croissantes de `nom` pour chaque parcours.
`x[order(x$parcours,x$nom, decreasing=TRUE),]`	Tri des lignes de `x` par valeurs décroissantes de la colonne `parcours` puis suivant les valeurs décroissantes de `nom` pour chaque parcours.

Console R

```
> x[order(x$parcours),]
    nom note sexe parcours
2 Koffi   18    M      CHM
4  Adjo   11    F      CHM
1   Afi   17    F      MTH
5 Dodji   16    M      MTH
7 Abalo   20    M      MTH
3 Kodjo    9    M      PHY
6  Abla   12    F      PHY
> x[order(x$parcours,x$nom),]
    nom note sexe parcours
4  Adjo   11    F      CHM
2 Koffi   18    M      CHM
7 Abalo   20    M      MTH
1   Afi   17    F      MTH
5 Dodji   16    M      MTH
6  Abla   12    F      PHY
```

```
3 Kodjo    9    M      PHY
> x[order(x$parcours,x$nom,decreasing=TRUE),]
    nom note sexe parcours
3 Kodjo    9    M      PHY
6  Abla   12    F      PHY
5 Dodji   16    M      MTH
1   Afi   17    F      MTH
7 Abalo   20    M      MTH
2 Koffi   18    M      CHM
4  Adjo   11    F      CHM
```

2.8 VECTEURS LOGIQUES (CLASSE logical)

Un vecteur logique est un vecteur dont chaque composante prend l'une des valeurs logiques ou valeurs de vérité **TRUE** (Vrai) et **FALSE** (Faux).

2.8.1 Création par saisie

Exemple 2.6 On interroge dix étudiants d'un cours de Statistique sur une question de type Vrai (V) ou Faux (F). Les réponses obtenues sont les suivantes :

F V F V F F F V V V

On peut saisir les données et les afficher :

Console R

```
> x=c(FALSE,TRUE,FALSE,TRUE,FALSE,FALSE,FALSE,TRUE,TRUE,TRUE)
> print(x)
 [1] FALSE  TRUE FALSE  TRUE FALSE FALSE FALSE  TRUE  TRUE  TRUE
```

S'il n'existe pas de variables créées par l'utilisateur et nommées `F` et `T`, alors l'on peut aussi saisir les données comme suit :

Console R

```
> x=c(F,T,F,T,F,F,F,T,T,T)
> print(x)
 [1] FALSE  TRUE FALSE  TRUE FALSE FALSE FALSE  TRUE  TRUE  TRUE
```

2.8.2 Opérations sur les vecteurs logiques

`as.numeric(FALSE) ;` `as.numeric(TRUE)`	Respectivement 0 et 1.

I. C. GERALDO

`sum(x)`	Lorsqu'on applique une fonction numérique à un vecteur logique, les `FALSE` et les `TRUE` deviennent respectivement 0 et 1. Ainsi, cette commande permet de compter le nombre de fois que `TRUE` apparaît dans `x`.
`table(x)`	Tableau de contingence de `x`.

2.8.3 Opérateurs de comparaison

`x=c(3,-2,5,0,7)` `print(x>2)`	Affichage d'un vecteur de même longueur que `x` avec des `TRUE` correspondant aux composantes de `x` strictement plus grandes que 2 et des `FALSE` ailleurs.
`x>=2 ; x<2 ; x<=2 ; x==2 ; x!=2`	Autres exemples de comparaison : inégalités larges, égalité, différence.

Console R

```
> x=c(3,-2,5,0,7)
> print(x>2)
[1]  TRUE FALSE  TRUE FALSE  TRUE
```

2.8.4 Opérateurs (connecteurs) logiques

Soit `x` un vecteur numérique.

`(x>2) & (x<=6)`	Conjonction (« et » logique). Retourne TRUE pour les éléments de `x` vérifiant les deux conditions et FALSE pour les autres.
`(x>2) | (x<=6)`	Disjonction (« ou »). Retourne TRUE pour les éléments de `x` vérifiant au moins l'une des deux conditions et FALSE pour les autres.
`!(x==5)`	Négation.
`xor(x>2,x<=6)`	« ou » exclusif. Retourne TRUE pour les éléments de `x` vérifiant une seule des deux conditions et FALSE pour les autres.

I. C. GERALDO

Console R

```
> x=c(3,-2,5,0,7)
> (x>2) & (x<=6)
[1]  TRUE FALSE  TRUE FALSE FALSE
> (x>2) | (x<=6)
[1] TRUE TRUE TRUE TRUE TRUE
> !(x==5)
[1]  TRUE  TRUE FALSE  TRUE  TRUE
> xor(x>2,x<=6)
[1] FALSE  TRUE FALSE  TRUE  TRUE
```

2.8.5 Fonctions logiques usuelles

`exists("y")`	Retourne TRUE si la variable y existe et FALSE sinon.
`is.na(x)`	Retourne un vecteur ou une matrice de même taille que x indiquant pour chaque composante de x s'il s'agit d'une valeur manquante.
`is.finite(x)`	Retourne un vecteur ou une matrice de même taille que x indiquant pour chaque composante de x si elle est finie ou non (infinie).

`is.vector(x); is.numeric(x);`
`is.complex(x); is.matrix(x);`
`is.data.frame(x);`
`is.character(x); is.factor(x);`
`is.list(x); is.logical(x)`
	L'objet x est-il un vecteur (réel ou complexe) ? un vecteur réel ? un vecteur complexe ? une matrice ? un tableau de données ? un vecteur de chaînes de caractères ? un facteur ? une liste ? un vecteur de valeurs logiques ?
	Ces fonctions retournent une seule valeur logique.

`isSymmetric(M)`	Retourne TRUE si la matrice M est symétrique et FALSE sinon.
`is.element(x,y)` `x %in% y`	Commandes équivalentes retournant un vecteur de même longueur que x contenant TRUE pour les éléments de x appartenant à y et FALSE pour les autres.
`any(x>2)`	Retourne TRUE si au moins un élément de x est strictement supérieur à 2 et FALSE sinon.

`all(x>2)`	Retourne `TRUE` si tous les éléments de `x` sont strictement supérieurs à 2 et `FALSE` sinon.
`all(x==y)`	Commande permettant de déterminer si deux vecteurs `x` et `y` sont égaux.
`setequal(x,y)`	Commande permettant de déterminer si deux vecteurs `x` et `y` (pouvant être de longueurs différentes) contiennent les mêmes éléments lorsqu'on ne tient compte ni des doublons ni de l'ordre des éléments.
`60%in%x; is.element(60,x); any(x==60)`	Commandes équivalentes pour tester si 60 est une composante de `x`.
`duplicated(x)`	Retourne un vecteur de même longueur que `x` en indiquant, pour chaque composante, `TRUE` si c'est la première fois qu'elle apparait et `FALSE` sinon.
`any(duplicated(x))`	Retourne `TRUE` si `x` contient des doublons et `FALSE` sinon.

2.8.6 Sélection d'éléments dans un vecteur

`x=c(3,-1,0,10,12); which(x>=10)`	Positions des éléments de `x` supérieurs ou égaux à 10. Retourne un vecteur de deux éléments 4 et 5.
`x[x>2]` `subset(x,x>2)`	Commandes équivalentes permettant de récupérer les composantes de `x` strictement supérieures à 2.
`x[x>2]=1`	Remplace toutes les composantes de `x` strictement supérieures à 2 par 1.

2.8.7 Sélection d'éléments dans un tableau (matrice ou tableau de données)

Sans perte de généralités, soit `A` la matrice définie comme suit :

Console R

```
> A=matrix(-4:4,nrow=3,ncol=3)
> print(A)
     [,1] [,2] [,3]
[1,]   -4   -1    2
```

```
[2,]   -3    0    3
[3,]   -2    1    4
```

On sait que la sélection des éléments de A peut se faire sous l'une des trois formes A[idx1,idx2], A[idx1,] et A[,idx2], où idx1 et idx2 sont des vecteurs contenant respectivement les numéros des lignes et des colonnes à sélectionner. On peut remplacer ces vecteurs de numéros de lignes et de colonnes par des vecteurs logiques définis à partir des lignes et des colonnes. Dans ce cas, seules les lignes et/ou colonnes correspondant à la valeur TRUE seront sélectionnées.

`which(A[,2]>=0)`	Numéros des lignes pour lesquelles l'élément de la colonne 2 est supérieur ou égal à 0.
`A[A[,2]==0,]` `subset(A,A[,2]==0)`	Commandes équivalentes permettant de récupérer les lignes pour lesquelles la deuxième colonne est nulle.
`A[,A[1,]>0]`	Commande permettant de récupérer les colonnes pour lesquelles la première ligne est strictement positive.
`A[A>2]=5`	Remplace toutes les composantes de A strictement supérieures à 2 par 5.

I. C. GE

2.9 EXERCICES

Exercice 2.1 Soit $n \in \mathbb{N}^*$ et soient les quantités suivantes :

$$A_1 = \sum_{k=1}^{n} k^3, \qquad A_2 = \sum_{k=0}^{n} kC_n^k, \qquad A_3 = \prod_{k=1}^{n} \cos\left(\frac{\pi}{2^{k+1}}\right),$$
$$B_1 = \left(\frac{n(n+1)}{2}\right)^2, \qquad B_2 = n2^{n-1}, \qquad B_3 = \frac{1}{2^n \sin\left(\frac{\pi}{2^{n+1}}\right)}.$$

Pour $n = 50$, calculer chaque quantité en une seule commande et vérifier que pour tout $i = 1, 2, 3$, l'on a $A_i = B_i$.

Exercice 2.2

1) (a) Créer les vecteurs `x1` et `x2` contenant respectivement les multiples de 2 et les multiples de 3 compris entre 2 et 50. Combien y en a-t-il ?

   (b) Appliquer des opérations d'ensembles aux vecteurs `x1` et `x2` pour obtenir le vecteur `x3` des multiples communs à 2 et 3 et le vecteur `x4` des multiples de 3 qui ne sont pas multiples de 2.

2) On veut reprendre la question 1 en utilisant cette fois-ci des opérateurs logiques.

   (a) Créer le vecteur `x` des entiers de 2 à 50.

   (b) En utilisant les opérateurs logiques, extraire de `x` les vecteurs `x1`, `x2`, `x3` et `x4` contenant respectivement les multiples de 2, les multiples de 3, les multiples communs à 2 et 3 et les multiples de 3 qui ne sont pas multiples de 2.

Exercice 2.3 Le tableau ci-joint donne le pourcentage y de fruits avariés en fonction de la dose x d'un produit de conservation des fruits en grammes par litre.

x	5	7	10	12	15	17	19	20	23
y	8.5	8	6.5	6	5.5	5	3.5	3	2.5

1) Saisir les données dans deux vecteurs x et y.

2) Calculer le coefficient de corrélation linéaire entre x et y défini par :

$$R = \frac{\left(\frac{1}{n}\sum_{i=1}^{n} x_i y_i\right) - \left(\frac{1}{n}\sum_{i=1}^{n} x_i\right)\left(\frac{1}{n}\sum_{i=1}^{n} y_i\right)}{\sqrt{\left(\frac{1}{n}\sum_{i=1}^{n} x_i^2\right) - \left(\frac{1}{n}\sum_{i=1}^{n} x_i\right)^2} \times \sqrt{\left(\frac{1}{n}\sum_{i=1}^{n} y_i^2\right) - \left(\frac{1}{n}\sum_{i=1}^{n} y_i\right)^2}},$$

   où n est le nombre commun de composantes de x et y.

3) Comparer le résultat avec celui de la commande `cor(x,y)`.

Exercice 2.4 Ce exercice porte sur le jeu de données interne nommé **`precip`** contenant les quantités moyennes de précipitations en pouces (1 pouce = 2.54 cm) dans différentes villes américaines. Ecrire et exécuter le code R permettant de (d') :

I. C. GERALDO

1) charger et afficher le jeu de données `precip`;
2) afficher la classe de `precip`;
3) afficher la liste des villes concernées;
4) déterminer le nombre de villes concernées;
5) afficher le niveau de précipitation des villes suivantes : Philadelphia, Columbia, Baltimore.

Exercice 2.5 On considère la matrice carrée d'ordre 4 suivante :

$$M = \begin{pmatrix} 2 & 4 & 1 & 2 \\ 3 & 4 & 5 & 6 \\ 7 & 10 & 9 & 10 \\ 12 & 15 & 17 & 8 \end{pmatrix}.$$

1) Construire la matrice M.
2) Afficher :
   (a) l'élément situé à l'intersection de la première ligne et de la troisième colonne;
   (b) la première ligne;
   (c) la troisième colonne;
   (d) la sous-matrice obtenue après avoir enlevé la troisième ligne et la deuxième colonne;
   (e) la somme des éléments par lignes;
   (f) la sous-matrice obtenue en ne gardant que les lignes dont la somme est supérieure ou égale à 40;
   (g) la sous-matrice obtenue en ne gardant que les lignes commençant par un nombre pair.

I. C. GERALDO

Exercice 2.6 Ecrire et exécuter le code permettant de créer sans saisir les composantes,

1) la matrice `A` d'ordre 5×5 avec des 1 partout sauf des 0 sur la diagonale;
2) la matrice `B` d'ordre 5×5 avec les nombres 1 à 49 de deux en deux et répartis de façon croissante colonne par colonne.
3) La matrice `B2` issue de `B` où les multiples de 3 sont remplacés par la valeur 0.
4) La matrice `B3` issue de `B` où les éléments en dessous de la diagonale sont remplacés par la valeur 0.

Exercice 2.7 Selon l'annuaire statistique national 2022 du Togo publié par l'Institut National de la Statistique et des Etudes Economiques et Démographiques (INSEED), la répartition des étudiants inscrits en 2018 dans les universités et grandes écoles du Togo par cycle d'études et par sexe est la suivante :

Cycle d'études	Sexe	
	F	M
1er cycle	29 882	59 481
2e cycle	1 027	2 862
3e cycle	322	1 245

1) Saisir les données dans une matrice x telle qu'à l'affichage l'on ait exactement le résultat suivant :

Console R

```
> print(x)
              F     M
1er cycle 29882 59481
2e cycle   1027  2862
3e cycle    322  1245
```

2) Afficher x avec en marges les sommes par lignes et par colonnes.
3) Déterminer l'effectif total des étudiants en 2018.
4) Calculer le ratio Nombre de filles/Nombre de garçons par cycle puis pour tous les cycles confondus.

Exercice 2.8 Répondre aux questions ci-après en utilisant des commandes R.

1) Charger dans votre session R le jeu de données interne `WorldPhones` et afficher son contenu.
2) Que représente ce jeu de données ? Quelle est sa classe ?
3) Calculer le nombre total de téléphones pour chaque année (vecteur `nbtel.an`).
4) Quel est le continent qui a le plus de numéros attribués en 1961 ?
5) En 1961, dans combien de continents y a-t-il plus de 20 millions de téléphones ?
6) En quelle année, le nombre total de téléphones a-t-il dépassé 120 millions pour la première fois ?
7) Calculer pour chaque année, la proportion du nombre de téléphones de l'Afrique par rapport au nombre de téléphones dans le monde.

Exercice 2.9 On a relevé les informations sur les appartements disponibles auprès d'un agent immobilier et obtenu les données suivantes :

loyer	cuisine	nbch	quartier
27000	TRUE	1	AGOE
35000	TRUE	2	TOTSI
40000	TRUE	2	KEGUE
10000	FALSE	1	BE
15000	FALSE	1	TOTSI
30000	TRUE	2	AGOE
12000	FALSE	1	TOTSI

I. C. GERALDO

où **loyer** indique le loyer mensuel en francs CFA, **cuisine** indique la présence ou non d'une cuisine, **nbch** indique le nombre de chambres et **quartier** indique le quartier où se trouve l'appartement. Ecrire et exécuter le code R permettant de (d') :

1) saisir les données dans un tableau de données nommé **M** ;
2) déterminer le nombre d'appartements concernés ;
3) calculer le loyer moyen ;
4) calculer le loyer moyen par quartier ;
5) déterminer le nombre de quartiers concernés ;
6) déterminer le nombre d'appartements avec cuisine ;
7) afficher les données de l'appartement le plus cher ;
8) afficher les données de tous les appartements avec cuisine situés à TOTSI ;
9) ajouter à M, sans ressaisir M :
   - une colonne **caution** contenant la valeur 6 et correspondant au nombre de mois de loyer servant de caution,
   - une colonne **avance** contenant les valeurs 3, 4, 6, 2, 2, 4 et 2 correspondant au nombre de mois de loyer à payer en avance
   - et une colonne **loyer2** correspondant au loyer augmenté des charges additionnelles (par exemple, frais d'usage de fosse sceptique) s'élevant à 500 francs CFA pour tous les appartements.
10) afficher le tableau M en triant les appartements du plus cher au moins cher.

Exercice 2.10 Répondre aux questions ci-après en utilisant des commandes R.

1) Charger le jeu de données interne `airquality` et l'afficher.
2) Que représente ce jeu de données ?
3) Donner sa classe, ses variables (colonnes) et son nombre de lignes.
4) Donner le nombre et le pourcentage de valeurs manquantes pour chaque variable.
5) Créer, à partir de `airquality`, un tableau de données `x` dont les températures seront converties en degrés Celsius. On rappelle la formule de conversion de Fahrenheit en Celsius : $°C = (°F - 32)/1.8$.
6) Calculer, en degrés Celsius, les températures moyennes, minimales et maximales par mois.
7) Calculer la concentration moyenne d'ozone (faire attention aux données manquantes !).
8) Calculer la concentration moyenne d'ozone par mois.

I. C. GERALDO

CHAPITRE 3

STATISTIQUE DESCRIPTIVE

Sommaire

3.1 Séries statistiques à une variable 48
3.1.1 Variable quantitative 48
3.1.2 Variables qualitatives 58
3.2 Tableaux de contingence à double entrée 60
3.2.1 Généralités 60
3.2.2 Distributions et fréquences marginales 61
3.2.3 Profils lignes et profils colonnes 61
3.2.4 Représentation graphique 62
3.3 Ajustement linéaire 64
3.3.1 Nuage de points 64
3.3.2 Coefficient de corrélation linéaire 65
3.3.3 Ajustement linéaire 65
3.4 Exercices 68

I. C. GERALDO

3.1 SÉRIES STATISTIQUES À UNE VARIABLE

3.1.1 Variable quantitative

3.1.1.1 Indicateurs numériques

Soit `x` un vecteur numérique à n composantes dont les composantes sont ici notées x_1, ..., x_n.

`min(x); max(x)`	Minimum et maximum des composantes de `x`.
`range(x)`	Vecteur composé du minimum et du maximum des composantes de `x`.
`sum(x)`	Somme des composantes de `x`
`mean(x)`	Moyenne des composantes de `x`

Dans R, il existe une fonction `var`. Mais cette fonction appliquée à un vecteur `x` calcule plutôt la **variance corrigée** des composantes de `x` dont la formule est :

$$S_n'^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \overline{x}_n)^2, \tag{3.1}$$

où $\overline{x}_n$ désigne la moyenne des composantes de `x`. Pour calculer la variance usuelle dont la formule est

$$S_n^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \overline{x}_n)^2, \tag{3.2}$$

on utilise la propriété

$$S_n^2 = \left(\frac{1}{n}\sum_{i=1}^{n}x_i^2\right) - (\overline{x}_n)^2 = \text{moyenne des carrés} - (\text{moyenne})^2 \tag{3.3}$$

comme suit :

`mean(x^2)-(mean(x))^2`	Variance des composantes de `x`.
`weighted.mean(c(0,1,2),c(3,2,5))`	Moyenne pondérée de 0, 1 et 2 avec poids respectifs 3, 2 et 5. Elle correspond à $\frac{3\times 0+2\times 1+5\times 2}{3+2+5} = 1.2$
`median(x)`	Médiane des valeurs de `x`.
`quantile(x,0.25)`	Quantile d'ordre 0.25 des composantes de `x` (encore appelé premier quartile ou quartile Q_1).
`quantile(x,probs=c(0.25,0.75))`	Les quantiles d'ordre 0.25 et 0.75 respectivement (encore appelés quartiles).
`summary(x)`	Résumé des indicateurs numériques (moyenne, quartiles, minimum, etc.) sur le vecteur `x`. Cette fonction peut aussi être utilisée pour d'autres classes.
`table(x)`	Tableau de contingence de `x`. Ce tableau donne les différentes modalités et les effectifs associés à chaque modalité.

`prop.table(table(x))`
`proportions(table(x))`
`table(x)/length(x)`
`table(x)/sum(table(x))`
	Commandes équivalentes permettant d'obtenir le tableau des fréquences des composantes de `x`.

Exemple 3.1 On a relevé le nombre de frères et sœurs de 20 étudiants lors d'un cours de Statistique et l'on a obtenu les données suivantes :

2	4	3	2	2	1	1	3	2	4	4	0	3	3	3	4	4	0	1	3

1) On peut saisir les données à l'aide du code suivant :

```
x=c(2,4,3,2,2,1,1,3,2,4,4,0,3,3,3,4,4,0,1,3)
```

2) On peut afficher le tableau de contingence à l'aide du code suivant :

```
table(x)
```

dont le résultat est

```
x
0 1 2 3 4
2 3 4 6 5
```

La première ligne donne les modalités et la deuxième donne les effectifs associés à chaque modalité.

3) On peut afficher le tableau des fréquences à l'aide du code suivant :

```
prop.table(table(x))
```

dont le résultat est

```
x
   0    1    2    3    4
0.10 0.15 0.20 0.30 0.25
```

La première ligne donne les modalités et la deuxième donne les fréquences associées à chaque modalité.

3.1.1.2 Diagramme en barres

Soit `x` un vecteur numérique contenant les données.

`barplot(table(x))`	Affiche le diagramme en barres associé à `x`.
`barplot(table(x), xlab="...", ylab="...", col="...", border=NA)`	Affiche le diagramme en barres associé à `x`. L'argument **xlab** permet d'écrire du texte en dessous de l'axe des abscisses et l'argument **ylab** permet d'écrire du texte à gauche de l'axe des ordonnées. L'argument **col** permet de préciser la couleur. L'argument **border** permet de préciser la couleur de la bordure des barres (ici l'option **border=NA** supprime la bordure des barres).

I. C. GERALDO

Remarque 3.1 Dans R, il existe 657 couleurs prédéfinies. Leur liste est disponible grâce à la commande

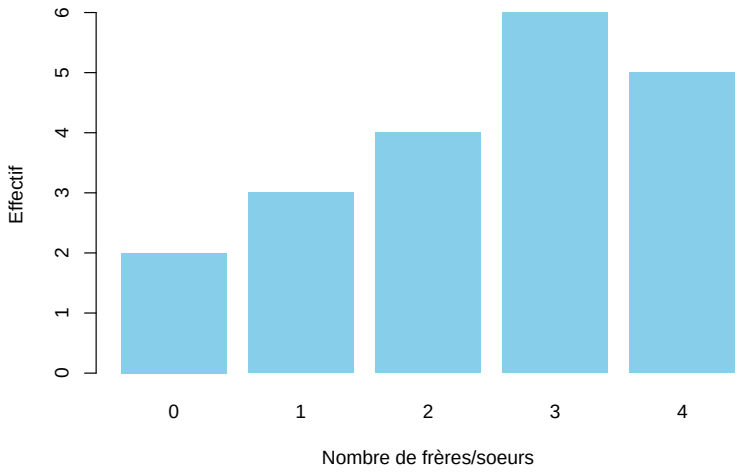
```
colors()
```

Parmi les noms de couleurs classiques, on note :

- "blue",
- "red",
- "green",
- "yellow",
- "gray",
- "lightgray",
- "yellowgreen",
- "lightblue",
- "skyblue",
- "lightgreen",
- "tomato".

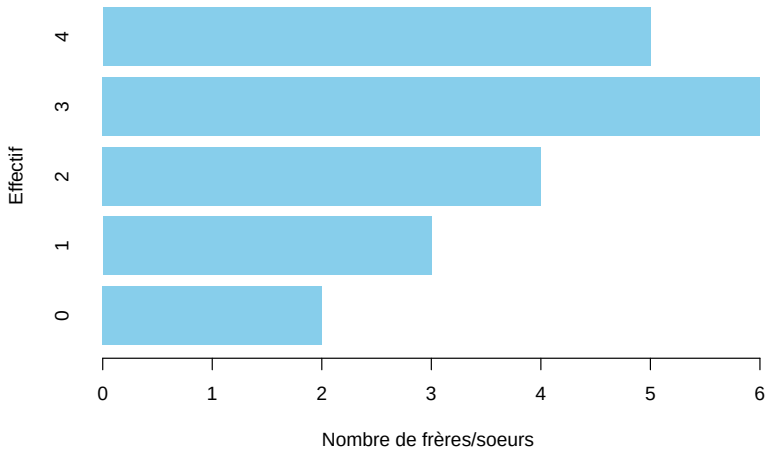
Exemple 3.1 (suite).

```
barplot(table(x), xlab="Nombre de frères/soeurs", ylab="Effectif",
        col="skyblue", border=NA)
```



On peut avoir des barres horizontales en ajoutant l'option **horiz=TRUE** comme suit :

```
barplot(table(x), xlab="Nombre de frères/soeurs", ylab="Effectif",
        col="skyblue", border=NA, horiz=TRUE)
```



3.1.1.3 Diagramme en bâtons

Soit x un vecteur numérique contenant les données.

```
plot(table(x),lwd=5,lend=2,las=1)
```

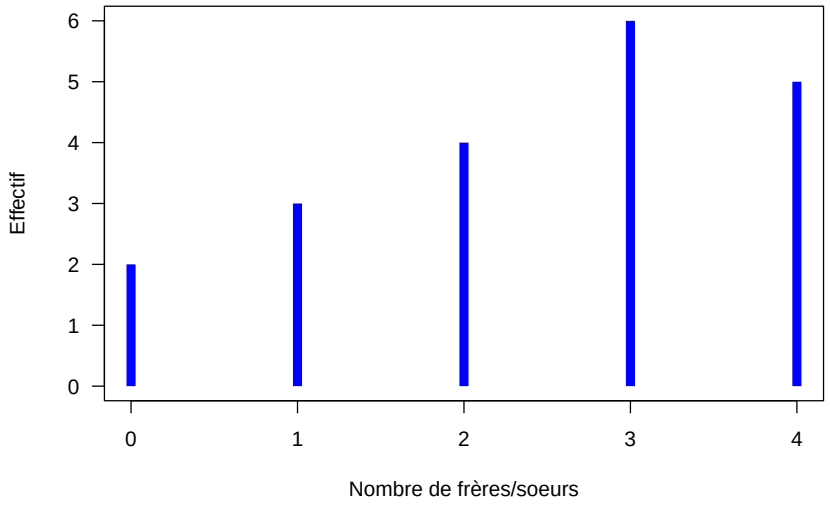
Construction du diagramme en bâtons associé à x.

- L'argument **lwd** (line width) est un entier naturel spécifiant la largeur de chaque trait.
- L'argument **lend** (line end) permet de préciser la forme des extrémités (0 = extrémité arrondie ou circulaire, 1 = extrémité plate ou carrée).
- L'argument **las** (entier naturel de valeur 0, 1, 2 ou 3) permet de préciser le style des étiquettes des axes : 0 (étiquettes parallèles aux axes, valeur par défaut), 1 (style horizontal), 2 (étiquettes perpendiculaires aux axes), 3 (style verticale). C'est un argument général valable pour toutes les fonctions de représentation graphique.

I. C. GERALDO

Exemple 3.1 (suite).

```
plot(table(x), lwd=7, lend=1, xlab="Nombre de frères/soeurs",
     ylab="Effectif", col="blue", las=1)
```




```
plot(x,y,type="h")
```

L'option `type="h"` permet aussi de construire un diagramme en bâtons dont `x` contient les modalités et `y` contient les effectifs.

3.1.1.4 Histogramme

L'histogramme permet de représenter une variable statistique continue groupée (définie par intervalles).

Exemple 3.2 On a noté l'âge (arrondi à l'année près) de 48 salariés d'une entreprise. La série statistique brute (données obtenues pendant l'enquête) est donnée ci-dessous :

43	29	57	45	50	29	37	59	46	31	46	24
33	38	49	31	62	60	52	38	43	26	41	52
60	49	52	41	38	26	37	59	57	41	29	33
33	43	46	57	46	33	46	49	57	57	46	43

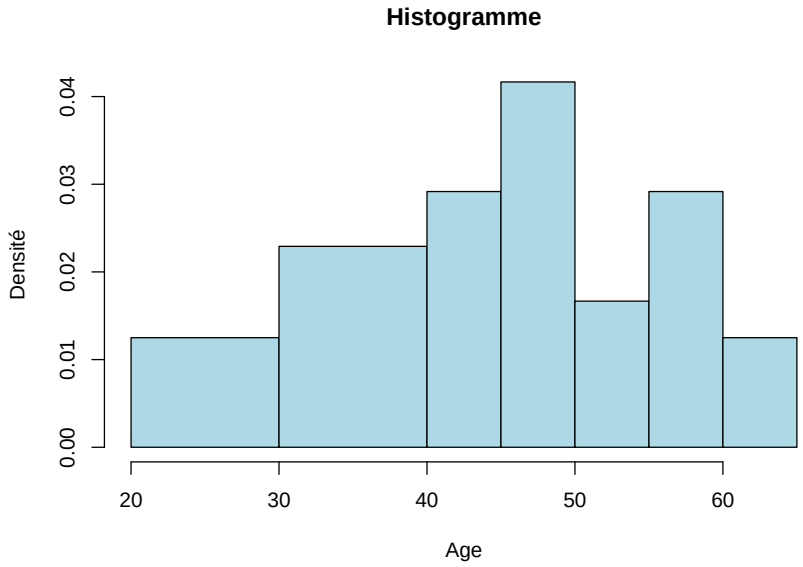
On peut saisir les données à l'aide de la commande suivante :

```
age = c(43, 29, 57, 45, 50, 29, 37, 59, 46, 31, 46, 24,
        33, 38, 49, 31, 62, 60, 52, 38, 43, 26, 41, 52,
        60, 49, 52, 41, 38, 26, 37, 59, 57, 41, 29, 33,
        33, 43, 46, 57, 46, 33, 46, 49, 57, 57, 46, 43)
```

I. C. GERALDO

L'**histogramme en fréquences** (ou en densité) du vecteur `x` en considérant les classes d'âges $[20, 30[$, $[30, 40[$, $[40, 45[$, $[45, 50[$, $[50, 55[$, $[55, 60[$ et $[60, 65]$ peut être obtenu à l'aide de la fonction **hist** comme suit :

```
h = hist(age, breaks = c(20, 30, 40, 45, 50, 55, 60, 65), right = FALSE,
         freq = FALSE, col="lightblue", xlab="Age", ylab="Densité",
         main="Histogramme")
```



- Par défaut, les classes sont semi-fermées à droite et la première est fermée (**`right = TRUE`**). L'argument **`right = FALSE`** permet de spécifier que les classes soient semi-fermées à gauche et que la dernière classe soit fermée.
- L'option **`freq = FALSE`** est obligatoire pour obtenir un histogramme au sens classique du terme c'est-à-dire un histogramme des fréquences.
- L'argument **`col`** précise la couleur, les arguments **`xlab`** et **`ylab`** précisent les textes à écrire respectivement en dessous de l'axe des abscisses et à gauche de l'axe des ordonnées et l'argument **`main="Histogramme"`** permet de préciser le titre du graphique.

`h$counts`	Les effectifs des classes.
`prop.table(h$counts)`	Les fréquences des classes.
`h$mids`	Les centres des classes.

`h=hist(age,freq=FALSE,breaks=5)`	Histogramme avec cinq (5) classes de même amplitude.

Remarque 3.2 Lorsqu'on veut juste une répartition des données en classes sans faire d'histogramme ni aucun calcul, on peut utiliser la fonction **cut** avec les arguments **breaks** et **right**.

```
cut(x,breaks=...,right=...)
```

Pour un vecteur numérique x, cette commande retourne un vecteur de facteurs de même taille que x dans lequel figurent les classes associées aux composantes de x. Le rôle des arguments breaks et right est le même que pour la fonction hist.

Console R

```
> x=1:10
> cut(x,breaks=c(0,3,7,10))
 [1] (0,3]  (0,3]  (0,3]  (3,7]  (3,7]  (3,7]  (7,10] (7,10] (7,10]
Levels: (0,3] (3,7] (7,10]
> table(cut(x,breaks=c(0,3,7,10)))

(0,3]  (3,7] (7,10]
    3      4      3
```

La parenthèse indique un intervalle semi-ouvert.

3.1.1.5 Boîtes à moustaches

```
boxplot(x)
```

- Si x est un vecteur numérique, cette commande affiche la boîte à moustaches associée à x.
- Si x est une liste dont les composantes sont des vecteurs numériques, cette commande affiche sur un même graphique, les boîtes à moustaches associées aux différentes composantes de x.
- Si x est une matrice, cette commande affiche sur un même graphique, les boîtes à moustaches associées aux différentes colonnes de x.

I. C. GERALDO

`boxplot(y∼x)`	Si y est un vecteur numérique et x est un vecteur de facteurs de même nombre de composantes que y, cette commande affiche sur un même graphique les différentes boîtes à moustaches associées aux valeurs de y scindées par modalités de x. L'expression y∼x, appelée **formule**, indique à `boxplot` de traiter la variable y en fonction de la variable x.
`boxplot(x,...,col="...")`	Définition de la couleur de remplissage des boîtes.
`boxplot(x,...,horizontal=TRUE)`	Les boîtes sont horizontales plutôt que verticales.
`boxplot(x,...,outline=TRUE)`	Les valeurs aberrantes sont affichées sur le graphique.
`boxplot(x,...,names=...)`	L'argument **names** (de classe **character**) permet de spécifier les noms à écrire sous chaque boîte.
`boxplot(x,...,boxwex=0.3)`	La largeur de chaque boîte est fixée à 30% de la largeur par défaut.
`help(boxplot)` `help(bxp)`	Ces deux commandes permettent d'avoir plus de détails sur le paramétrage de la fonction **boxplot**. La fonction **bxp** est celle qui est appelée à l'intérieur de la fonction `boxplot` pour tracer la boîte à moustaches, une fois les calculs faits.

Exemple 3.3 L'on a mesuré la hauteur (en mètres) de 30 arbres répartis en trois groupes de dix (G1, G2 et G3) et obtenu les résultats suivants :

G1	11.4	18.5	13.3	12.1	15.1	7.2	25.3	15.7	16.1	11.1
G2	12.4	15	23	13.6	9.1	10	23.5	16.3	16.9	12.7
G3	9.8	11.4	7.7	6	10.8	3.3	9.6	10.6	15.6	2.8

Voici trois exemples de codes permettant de représenter les données sous forme de boîtes à moustaches.

- Code 1 (Utilisation d'une liste)

```
h1=c(11.4, 18.5, 13.3, 12.1, 15.1, 7.2, 25.3, 15.7, 16.1, 11.1)
h2=c(12.4, 15, 23, 13.6, 9.1, 10, 23.5, 16.3, 16.9, 12.7)
h3=c(9.8, 11.4, 7.7, 6, 10.8, 3.3, 9.6, 10.6, 15.6, 2.8)
```

```
boxplot(list(h1,h2,h3), names=c("G1","G2","G3"), boxwex=0.4,
        outline=TRUE, xlab="Groupe", ylab="Hauteur (en m)", las=1)
```

- Code 2 (Utilisation d'une matrice)

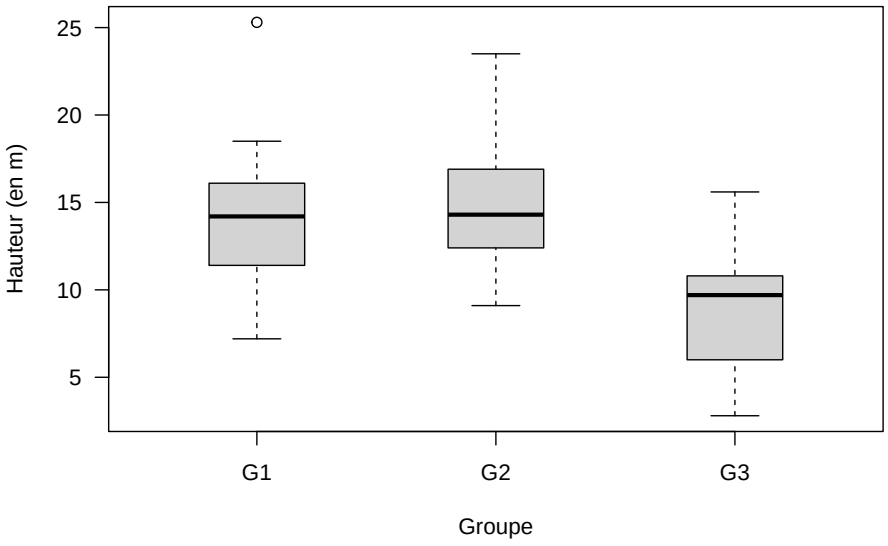
```
h1=c(11.4, 18.5, 13.3, 12.1, 15.1, 7.2, 25.3, 15.7, 16.1, 11.1)
h2=c(12.4, 15, 23, 13.6, 9.1, 10, 23.5, 16.3, 16.9, 12.7)
h3=c(9.8, 11.4, 7.7, 6, 10.8, 3.3, 9.6, 10.6, 15.6, 2.8)
h=matrix(c(h1,h2,h3),nrow=10)
colnames(h)=c("G1","G2","G3")
boxplot(h, boxwex=0.4, outline=TRUE, xlab="Groupe",
        ylab="Hauteur (en m)", las=1)
```

- Code 3 (Utilisation d'une formule)

```
h1=c(11.4, 18.5, 13.3, 12.1, 15.1, 7.2, 25.3, 15.7, 16.1, 11.1)
h2=c(12.4, 15, 23, 13.6, 9.1, 10, 23.5, 16.3, 16.9, 12.7)
h3=c(9.8, 11.4, 7.7, 6, 10.8, 3.3, 9.6, 10.6, 15.6, 2.8)
h=c(h1,h2,h3)
g=factor(rep(c("G1","G2","G3"),each=10))
boxplot(h~g, boxwex=0.4, outline=TRUE, xlab="Groupe",
        ylab="Hauteur (en m)", las=1)
```

L'écriture `h~g` est appelée une **formule** signifiant "h en fonction de g".

Les trois codes donnent le même graphique ci-après :



I. C. GERALDO

3.1.2 Variables qualitatives

Soit x un vecteur de classe **factor** ou **character** ou **logical** considéré comme un vecteur d'observations d'une variable qualitative.

3.1.2.1 Tableau de contingence et fréquences

`table(x)`	Tableau de contingence pour les observations contenues dans le vecteur `x`.
`freq=prop.table(table(x))` `freq=proportions(table(x))`	Commandes équivalentes permettant d'obtenir le tableau des fréquences.

3.1.2.2 Diagramme circulaire

`pie(freq)`	Affiche un camembert (diagramme circulaire) dont les fréquences sont contenues dans le vecteur `freq`.

Exemple 3.4 On a relevé la région de provenance de 15 étudiants participant à un cours de Statistique et obtenu les données suivantes :

KARA	SAVANES	SAVANES	KARA	CENTRALE
CENTRALE	CENTRALE	KARA	SAVANES	KARA
KARA	MARITIME	MARITIME	KARA	SAVANES

- On peut saisir les données à l'aide du code suivant :

```
region=factor(c("KARA","SAVANES","SAVANES","KARA","CENTRALE",
                "CENTRALE","CENTRALE","KARA","SAVANES","KARA",
                "KARA","MARITIME","MARITIME","KARA","SAVANES"))
```

ou

```
region=factor(c("K","S","S","K","C","C","C","K","S","K","K","M","M","K","S"),
              levels=c("C","K","M","S"),
              labels=c("CENTRALE", "KARA","MARITIME","SAVANES"))
```

- On peut afficher le tableau de contingence à l'aide du code

```
table(region)
```

dont le résultat est :

```
region
CENTRALE     KARA MARITIME  SAVANES
       3        6        2        4
```

I. C. GERALDO

- On peut afficher le tableau des fréquences à l'aide du code

```
freq=prop.table(table(region))
print(freq)
```

dont le résultat est :

```
region
 CENTRALE      KARA  MARITIME   SAVANES
0.2000000 0.4000000 0.1333333 0.2666667
```

- On peut arrondir le tableau des fréquences à deux chiffres après la virgule à l'aide du code

```
freq=round(prop.table(table(region)),2)
print(freq)
```

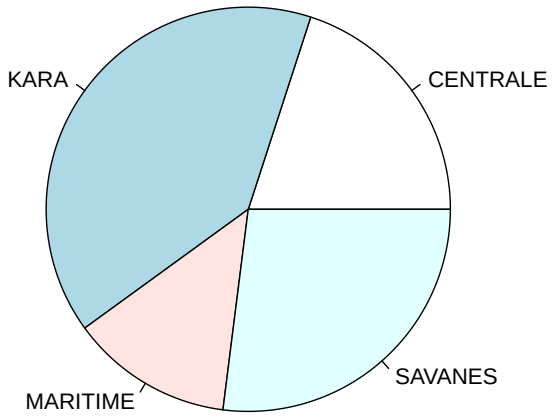
dont le résultat est :

```
region
CENTRALE      KARA MARITIME  SAVANES
    0.20      0.40      0.13      0.27
```

- On peut afficher le diagramme circulaire à l'aide du code

```
pie(freq)
```

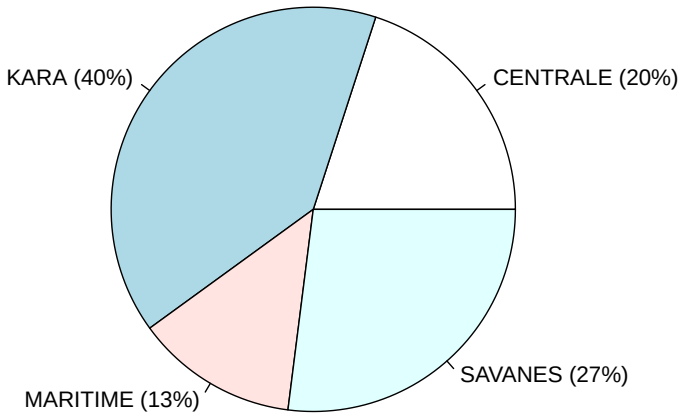
dont le résultat est :



On peut afficher les fréquences entre parenthèses sur le diagramme en utilisant l'argument **labels** qui permet de donner le texte à afficher sur le diagramme et la commande **paste** qui permet de coller des chaînes de caractères.

I. C. GERALDO

```
pie(freq,labels=paste(names(freq)," (",100*freq,"%)",sep=""))
```



3.2 TABLEAUX DE CONTINGENCE À DOUBLE ENTRÉE

3.2.1 Généralités

On peut obtenir un tableau de contingence à double entrée en appliquant la fonction `table` sur deux vecteurs de données brutes de même longueur ou en saisissant directement le tableau.

Exemple 3.5 Considérons la série et la mention obtenue par 10 étudiants ayant réussi au BAC.

Série	A4	C	D	D	C	D	A4	C	D	D
Mention	AB	P	AB	TB	B	B	B	AB	TB	B

On peut saisir ces données et obtenir le tableau de contingence comme suit :

Console R

```
> mention=factor(c("AB","P","AB","TB","B","B","B","AB","TB","B"),
+ levels=c("P","AB","B","TB"))
> serie=factor(c("A4","C","D","D","C","D","A4","C","D","D"))
>
> print(table(serie,mention))
     mention
serie P AB B TB
   A4 0  1 1  0
   C  1  1 1  0
   D  0  1 2  2
```

I.C.GERA

Exemple 3.6 En 2022, l'effectif de la population togolaise en fonction du milieu de résidence et du sexe est donné comme suit (source : https://inseed.tg/) :

Milieu de résidence	**Sexe**		**Total**
	Masculin	Féminin	
Urbain	1 689 674	1 784 118	3 473 792
Rural	2 254 836	2 366 870	4 621 706
Total	3 944 510	4 150 988	8 095 498

On peut saisir ce tableau (sans les totaux) à l'aide d'une matrice dont les lignes et colonnes seront nommées comme suit :

Console R

```
> tab=matrix(c(1689674, 1784118, 2254836, 2366870), nrow=2, byrow=TRUE)
> rownames(tab)=c("Urbain","Rural")
> colnames(tab)=c("Masculin","Féminin")
>
> print(tab)
       Masculin Féminin
Urbain  1689674 1784118
Rural   2254836 2366870
```

I. C. GERALDO

3.2.2 Distributions et fréquences marginales

On peut obtenir les distributions et fréquences marginales en utilisant les fonctions `addmargins`, `rowSums`, `colSums` (voir chapitre 2). Par exemple,

Console R

```
> addmargins(tab)
       Masculin Féminin     Sum
Urbain  1689674 1784118 3473792
Rural   2254836 2366870 4621706
Sum     3944510 4150988 8095498
>
> proportions(rowSums(tab))
   Urbain     Rural
0.4291017 0.5708983
>
> proportions(colSums(tab))
 Masculin   Féminin
0.4872474 0.5127526
```

3.2.3 Profils lignes et profils colonnes

Soit `M` un tableau de contingence à double entrée.

`proportions(M,1)` `prop.table(M,1)`	Commandes équivalentes permettant d'obtenir les profils lignes (tableau obtenu en divisant chaque effectif par l'effectif total de la ligne correspondante).
`proportions(M,2)` `prop.table(M,2)`	Commandes équivalentes permettant d'obtenir les profils colonnes (tableau obtenu en divisant chaque effectif par l'effectif total de la colonne correspondante).

Console R

```
> round(proportions(tab,1),3)
       Masculin Féminin
Urbain    0.486   0.514
Rural     0.488   0.512
>
> round(proportions(tab,2),3)
       Masculin Féminin
Urbain    0.428    0.43
Rural     0.572    0.57
```

I. C. GERALDO

L'interprétation du tableau des profils lignes se fait ligne par ligne alors que celle des profils colonnes se fait colonne par colonne. Par exemple, du tableau des profils lignes, on peut déduire qu'en milieu urbain, les hommes et les femmes représentent respectivement 48.6% et 51.4% de la population alors qu'en milieu rural, les pourcentages d'hommes et de femmes sont respectivement de 48.8% et 51.2%.

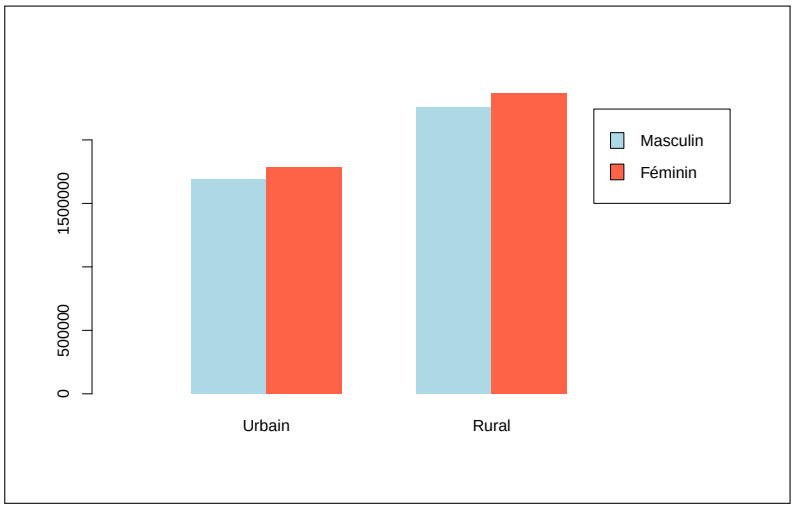
3.2.4 Représentation graphique

On peut faire, à l'aide la fonction `barplot`, un diagramme à barres juxtaposées ou à barres empilées.

- Voici un exemple de code donnant un diagramme à barres juxtaposées.

```
barplot(t(tab), beside=TRUE, col=c("lightblue","tomato"),
        border=NA, legend.text = colnames(tab), xlim=c(0,8))
```

Le résultat est :



La juxtaposition des barres est spécifiée par l'option `beside=TRUE`. L'ajout de l'option `xlim=c(0,8)` permet d'étendre les limites de l'axe des abscisses de sorte que la légende ne se colle pas au graphique.

- Voici un exemple de code donnant un diagramme à barres empilées.

```
barplot(t(tab), beside=FALSE, col=c("lightblue","tomato"),
                border=NA, legend.text = colnames(tab), xlim=c(0,3.5))
```

Cette fois, on a fixé `beside=FALSE`. Le résultat est :



3.3 AJUSTEMENT LINÉAIRE

Soient x et y deux vecteurs de n composantes contenant des données numériques.

3.3.1 Nuage de points

```
plot(x,y)
plot(y~x)
```

Commandes équivalentes permettant d'obtenir le nuage de points avec la variable x en abscisses et la variable y en ordonnées.

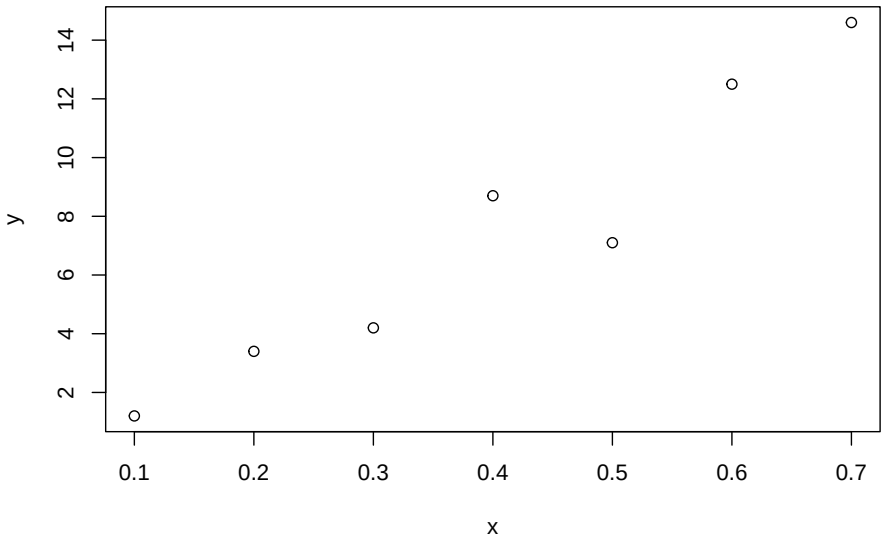
Exemple 3.7 Considérons deux variables statistiques x et y dont les observations sont données par le tableau suivant :

x	0.1	0.2	0.3	0.4	0.5	0.6	0.7
y	1.2	3.4	4.2	8.7	7.1	12.5	14.6

On peut saisir les données et représenter le nuage de points à l'aide du code suivant :

```
x = c(0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7)
y = c(1.2, 3.4, 4.2, 8.7, 7.1, 12.5, 14.6)
plot(x,y)
```

On obtient comme résultat :



On peut changer le symbole utilisé pour représenter chaque point du nuage à l'aide du paramètre `pch` (plotting character) qui prend comme valeur un entier compris entre 0 et 25.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
□	○	△	+	×	◇	▽	⊠	✳	⊕	⊕	⊠	⊞	⊠	⊡	■	●	▲	◆	●	•	○	□	◇	△	▽

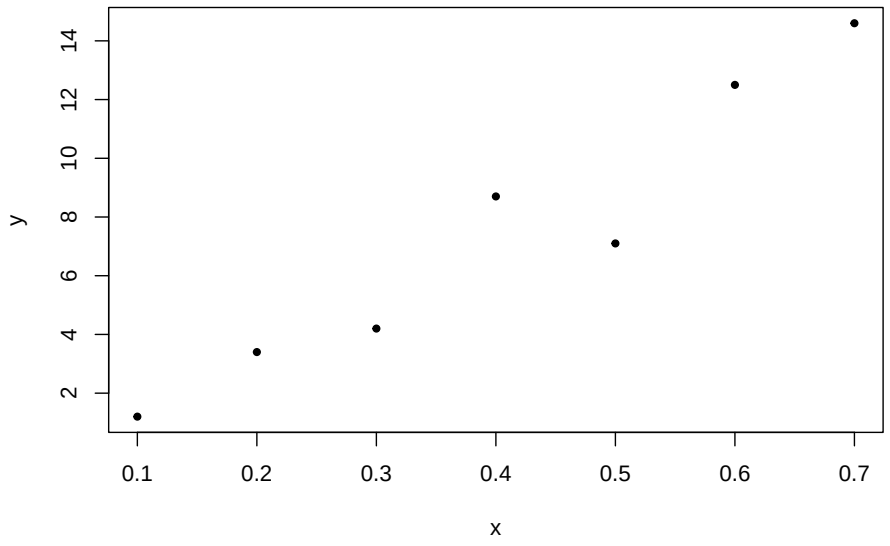
TABLE 3.1 – Valeurs du paramètre `pch` et symboles correspondants

Le tableau 3.1 donne sur la première ligne les différentes valeurs du paramètre `pch` et, sur la deuxième ligne, les symboles associés à ces différentes valeurs.

Ainsi, le code

```
plot(x,y,pch=20)
```

donne comme résultat :



3.3.2 Coefficient de corrélation linéaire

`cor(x,y)`	Coefficient de corrélation linéaire entre `x` et `y`.

3.3.3 Ajustement linéaire

L'ajustement linéaire par la méthode des moindres carrés ordinaires (MCO) se fait grâce à la fonction `lm` (signifiant *linear model*) comme suit :

```
reg=lm(y~x)
```

Ajustement linéaire de la forme

$$y = ax + b.$$

L'estimation des coefficients a et b est faite par la méthode MCO.

L'objet **reg** contient alors les résultats de l'ajustement linéaire. Pour afficher les valeurs des coefficients a et b, on peut utiliser le code suivant :

```
print(reg$coefficients)
```

qui donne comme résultat :

```
(Intercept)           x
  -1.371429   21.892857
```

L'objet **reg$coefficients** est un vecteur numérique à deux composantes :

- la première désignée par le terme **Intercept** représente la constante c'est-à-dire le coefficient b ;
- la seconde composante est le coefficient qui multiplie x c'est-à-dire le coefficient a.

On a donc

$$a = 21.892857 \qquad \text{et} \qquad b = -1.371429.$$

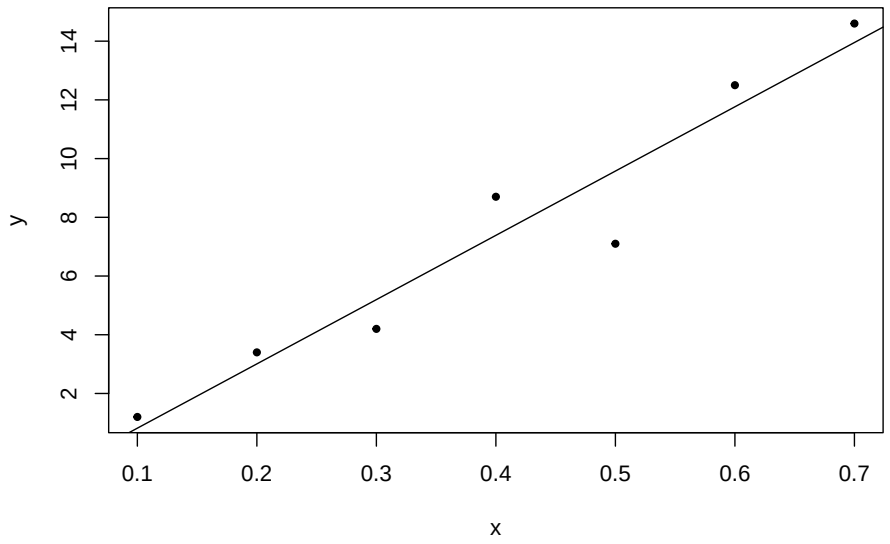
```
a=reg$coefficients[2]
b=reg$coefficients[1]
```

Récupération des coefficients du modèle.

On peut tracer la droite de régression sur le nuage de points en utilisant la fonction **abline** comme suit :

```
abline(reg$coefficients)
```

I. C. GERALDO




```
(cor(x,y))^2
```

Coefficient de détermination R^2 (égal au carré du coefficient de corrélation linéaire entre x et y).

Ici, on obtient $R^2 \approx 0.93$.

I. C. GERALDO

3.4 EXERCICES

Exercice 3.1 On a relevé le nombre d'enfants de 24 couples. Les données relevées sont les suivantes :

2	2	0	2	4	3	2	1	2	1	0	0
0	2	2	1	3	0	1	2	1	0	2	2

1) Saisir les données dans un vecteur nommé `x`.
2) Faire le tableau de contingence et calculer les fréquences.
3) Représenter les données par un diagramme en bâtons.
4) Calculer la moyenne, la variance et l'écart-type du nombre d'enfants.

Exercice 3.2 En 2022, l'effectif de la population togolaise est donné par région comme suit (source : https://inseed.tg/resultats-definitifs-du-rgph-5-novembre-2022/) :

Région	Nb. hommes	Nb. Femmes
Grand Lomé (GL)	1 060 504	1 127 872
Maritime sans GL	642 876	703 739
Plateaux	806 154	829 792
Centrale	397 336	398 193
Kara	488 225	497 287
Savanes	549 415	594 105

1) Saisir les données dans une matrice nommée `pop` dont les lignes et colonnes seront nommées telles qu'à l'affichage, l'on ait le résultat suivant :

```
                 Hommes  Femmes
Grand Lomé (GL)  1060504 1127872
Maritime sans GL  642876  703739
Plateaux          806154  829792
Centrale          397336  398193
Kara              488225  497287
Savanes           549415  594105
```

2) Calculer l'effectif total de la population togolaise en 2022.
3) Afficher la matrice `pop` en y ajoutant les marges (les totaux par lignes et par colonnes).
4) Calculer les fréquences par région (arrondies à trois chiffres après la virgule) puis les représenter à l'aide d'un diagramme circulaire sur lequel figureront les noms des régions et les fréquences en %.
5) Calculer les proportions d'hommes et de femmes (arrondies à trois chiffres après la virgule) pour la population totale puis par région.
6) Afficher à nouveau la matrice `pop` en classant les régions par ordre décroissant des effectifs et en y ajoutant les marges.
7) Afficher pour chaque région, son rang dans le classement des régions par ordre décroissant des effectifs.

I. C. GERALDO

Exercice 3.3 On a relevé la distance de freinage y d'un véhicule (distance parcourue par le véhicule entre le moment où le conducteur commence à freiner et l'arrêt du véhicule) à décélération constante sur une route plane sèche pour différentes vitesses x en km/h. Les données sont consignées dans le tableau suivant :

x (en km/h)	30	50	90	110	130
y (en m)	5.1	14.1	45.5	68	95

Les experts affirment que la distance d'arrêt d'un véhicule est proportionnelle à son énergie cinétique et donc au carré de sa vitesse. On cherche à confirmer cela à l'aide d'un ajustement par une fonction puissance de la forme

$$y = kx^{\alpha},$$

où k et α sont des constantes à déterminer.

1) Saisir les données et faire le nuage de points représentant les données.
2) Montrer en faisant un changement de variable simple, qu'on peut se ramener à un problème d'ajustement linéaire puis calculer les coefficients k et α par la méthode MCO.
3) La valeur obtenue pour α confirme-t-elle l'affirmation des experts ?
4) Confirmer la qualité du modèle en calculant le coefficient de corrélation linéaire entre y et x^2.
5) Estimer la distance de freinage correspondant à une vitesse de 70 km/h.
6) En utilisant la fonction `curve`, ajouter une représentation de la fonction d'expression $f(x) = kx^{\alpha}$ sur le nuage de points.

I. C. GERALDO

CHAPITRE 4

PROGRAMMATION DANS R

Sommaire

4.1 Structures de contrôle 70
4.1.1 Exécution conditionnelle avec `if` 70
4.1.2 Boucle `for` 72
4.1.3 Boucles `while` et `repeat` 73
4.2 Création de nouvelles fonctions 75
4.2.1 Création 75
4.2.2 Appel de la fonction 75
4.2.3 Affichage du code et des arguments d'une fonction 75
4.2.4 Sorties multiples 76
4.2.5 Arguments par défaut 76
4.2.6 Gestion des erreurs sur les arguments 76
4.2.7 Fonctions vectorisées (ou vectorielles) 77
4.3 Exercices 79

I.C. GERALDO

4.1 STRUCTURES DE CONTRÔLE

4.1.1 Exécution conditionnelle avec `if`

Il existe trois façons d'utiliser l'exécution conditionnelle avec **if**. On peut utiliser la syntaxe

Syntaxe R

```
if(condition)
{
    # commandes à exécuter si condition est vraie
}
```

ou la syntaxe

Syntaxe R

```
if(condition)
{
    # commandes à exécuter si condition est vraie
} else {
    # commandes à exécuter sinon
}
```

ou encore la syntaxe

Syntaxe R

```
if(condition1)
{
    # commandes à exécuter si condition1 est vraie
} else if(condition2) {
    # commandes à exécuter si condition1 est fausse et condition2 est vraie
} else if(condition3) {
    # commandes à exécuter si condition1 et condition2 sont fausses et
    # condition3 est vraie
} else {
    # commandes à exécuter si toutes les conditions sont fausses
}
```

où `condition`, `condition1`, etc... n'ont qu'une seule valeur logique `TRUE` ou `FALSE` (si ce sont des vecteurs logiques de longueur supérieure ou égal à 2, seul la première composante de chaque vecteur sera utilisée par R).

Remarque 4.1 Les accolades permettent de délimiter une commande ou un ensemble (bloc) de commandes.

Remarque 4.2 Lorsqu'elles sont utilisées, les commandes `else` et `else if` doivent être placées **obligatoirement** sur la même ligne que l'accolade fermante de `if` ou du `else if` précédent.

Remarque 4.3 Même si l'on s'est limité à deux `else if` dans la syntaxe générale ci-dessus, on peut mettre autant de `else if` que nécessaire.

Exemple 4.1 Le code suivant

```
x=3
if(x%%2==0)
{
    print("x est pair")
}
```

est correct mais n'affiche aucun résultat. En effet, l'opération `x%%2` donne le reste de la division euclidienne de `x` par 2. La condition `x%%2==0` permet de tester si ce reste est nul. Ici, le reste de la division euclidienne de 3 par 2 est égal à 1 et comme 1 est différent de 0, la condition `x%%2==0` vaut `FALSE` et donc la commande entre accolades n'est pas exécutée.

Exemple 4.2 Le code suivant

```
x=3
if(x%%2==0)
{
    print("x est pair")
} else {
    print("x est impair")
}
```

I. C. GERALDO

affiche le résultat suivant :

```
[1] "x est impair"
```

Exemple 4.3 Le code suivant

```
note=15
if(note<10){
        print("Résultat : Echec")
} else if(note<12){
        print("Résultat : Admis avec mention Passable")
} else if(note<14){
        print("Résultat : Admis avec mention Assez-Bien")
} else if(note<16){
        print("Résultat : Admis avec mention Bien")
} else {
        print("Résultat : Admis avec mention Très-Bien")
}
```

affiche le résultat suivant :

```
[1] "Résultat : Admis avec mention Bien"
```

4.1.2 Boucle `for`

Elle permet d'exécuter un bloc de commandes pour des valeurs fixées d'une variable appelée **compteur**. Sa syntaxe est la suivante :

Syntaxe R

```
for(compteur in vecteur)
{
    # commandes à exécuter
}
```

où `compteur` représente le nom de la variable compteur et `vecteur` est un vecteur contenant les différentes valeurs du compteur.

Exemple 4.4 Le code suivant

```
for(i in 1:5){print(i)}
```

donne comme résultat :

```
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
```

I. C. GERALDO

Il est possible de stopper la boucle prématurément (avant que le compteur n'ait parcouru toutes les valeurs) en utilisant respectivement la commande `break`.

Exemple 4.5 Le code suivant

```
for(i in 1:5)
{
    print(i)
    if(i==3){break}
}
```

donne comme résultat :

```
[1] 1
[1] 2
[1] 3
```

Dans une boucle `for`, il est aussi possible de sauter certaines valeurs du compteur et passer aux valeurs suivantes grâce à la commande `next`.

Exemple 4.6 Le code suivant

```
for(i in 1:5)
{
    if(i==3){next}
    print(i)
}
```

donne comme résultat :

```
[1] 1
[1] 2
[1] 4
[1] 5
```

I. C. GERALDO

4.1.3 Boucles `while` et `repeat`

La boucle `while` permet d'exécuter un bloc de commandes tant qu'une condition est vraie. Sa syntaxe est la suivante :

Syntaxe R

```
while(condition)
{
    # commandes à exécuter
}
```

Exemple 4.7 Le code suivant

```
i=1
while(i<=5){
    print(i)
    i=i+1
}
```

donne comme résultat

```
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
```

Remarque 4.4 Il faut s'assurer que parmi les commandes à exécuter à l'intérieur de la boucle `while`, il y a en ait une qui permette de mettre à jour la (les) variable(s) impliquées dans la condition sinon l'on risque d'avoir une boucle infinie (une boucle qui ne s'arrête jamais). Dans l'exemple 4.7, l'on peut remarquer que la commande `i=i+1` permet de mettre à jour la valeur de la variable `i` afin d'éviter la boucle infinie. Si l'on ne l'avait pas mise, la variable resterait bloquée à 1 et comme 1 est inférieur ou égal à 5, la boucle va afficher des 1 de façon infinie (sans s'arrêter).

La boucle `repeat`, quant à elle, permet de répéter un bloc de commandes jusqu'à ce qu'une condition soit vraie. Voici comment obtenir le même résultat que dans l'exemple 4.7 en utilisant la boucle `repeat`. Le code suivant

```
i=1
repeat{
    print(i)
    i=i+1
    if(i>5){break}
}
```

donne aussi comme résultat

```
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
```

Dans une boucle `repeat`, il est indispensable de vérifier la condition d'arrêt (ou de sortie) en plus de la mise à jour de la (des) variable(s) impliquées dans ladite condition. L'usage de `break` permet de sortir de la boucle dès que la condition de sortie est vérifiée.

I. C. GERALDO

4.2 CRÉATION DE NOUVELLES FONCTIONS

Une fonction est un objet permettant de stocker un ensemble de commandes. Il existe de nombreuses fonctions prédéfinies dans R que nous avons déjà vues dans les chapitres précédents (`c`, `matrix`, `list`, `data.frame`, `table`, `factor`, `cos`, `sin`, `mean`, etc...). Il est également possible pour un utilisateur de créer de nouvelles fonctions.

4.2.1 Création

```
somme = function(a,b)
{
    return(a+b)
}
```

Création d'une fonction `somme` permettant de calculer la somme des deux nombres `a` et `b` donnés en arguments.

```
somme=function(a,b){a+b}
```

Commande équivalente permettant de créer la même fonction `somme`.
Lorsque l'on n'utilise pas de `return`, c'est la dernière valeur figurant dans le corps de la fonction (ici `a+b`) qui est retournée comme résultat.

4.2.2 Appel de la fonction

```
somme(2,5)
somme(a=2,b=5)
somme(b=5,a=2)
```

Appels équivalents de la fonction `somme` avec `a=2` et `b=5`. Le résultat est 7.

4.2.3 Affichage du code et des arguments d'une fonction

```
print(somme)
```

Affiche le code de la fonction `somme`.

```
args(somme)
```

Affiche les noms des arguments de la fonction `somme`.

I. C. GE

4.2.4 Sorties multiples

```
somme.prod = function(a,b)
{
   return(list(somme=a+b,
               produit=a*b))
}
```

Lorsqu'une fonction doit retourner plusieurs valeurs de même classe ou de classes différentes, lesdites valeurs peuvent être toutes stockées en un seul objet de type liste. Ici, la fonction renvoie une liste de deux composantes (`somme` et `produit`).

```
somme.prod(2,4)$somme
```

Renvoie 6.

```
somme.prod(2,4)$produit
```

Renvoie 8.

4.2.5 Arguments par défaut

```
somme = function(a,b=0)
{
   return(a+b)
}
```

L'argument `b` a 0 comme valeur par défaut c'est-à-dire que si `b` n'est pas spécifié au moment de l'appel de la fonction, il sera mis à 0.

```
somme(1,0) ; somme(1)
```

Commandes équivalentes donnant comme résultat 1.

4.2.6 Gestion des erreurs sur les arguments

Dans l'écriture d'une fonction, il peut être nécessaire de vérifier si les arguments vérifient certaines conditions (par exemple, la positivité). Dans ce cas, on peut utiliser la commande **stop** qui permet d'afficher un message d'erreur et d'arrêter l'exécution du code de la fonction.

Exemple 4.8 Considérons le code suivant :

```
f = function(x)
{
    if(x<=0){stop("L'argument doit être strictement positif.")}
    return(1/sqrt(x))
}
```

Exécutons ce code pour des valeurs positives et négatives de l'argument. On a par exemple :

Console R

```
> f(4)
[1] 0.5
> f(7)
[1] 0.3779645
> f(0)
Error in f(0) : L'argument doit être strictement positif.
```

```
> f(-3)
Error in f(-3) : L'argument doit être strictement positif.
```

4.2.7 Fonctions vectorisées (ou vectorielles)

Une fonction est dite vectorisée (ou vectorielle) si lorsqu'on lui donne comme argument un vecteur x, elle retourne un vecteur de même taille que x contenant les images respectives des composantes de x par ladite fonction. Par exemple, les fonctions usuelles telles que `cos`, `sin`, `exp`, `log`, etc... sont vectorisées. Cette propriété est très utile notamment lors des représentations graphiques et du calcul d'intégrales.

Exemple 4.9 Voici un exemple de fonction vectorisée définie par l'utilisateur.

Console R

```
> f = function(x){return(2*x-cos(x))}
> f(0:5)
[1] -1.000000  1.459698  4.416147  6.989992  8.653644  9.716338
```

Considérons maintenant la fonction de $\mathbb{R}$ vers $\mathbb{R}$ définie par

$$g(x) = \frac{\sin x}{x} \quad \text{si} \quad x \neq 0 \quad \text{et} \quad g(0) = 0.$$

Le code de g et un exemple d'application sur un vecteur sont donnés ci-après.

Console R

```
> g = function(x){
+     if(x==0){y=0} else {y=sin(x)/x}
+     return(y)
+ }
>
> g(0:3)
[1] 0
Message d'avis :
Dans if (x == 0) { :
  la condition a une longueur > 1 et seul le premier élément est utilisé}
```

L'erreur est due au fait que la fonction g n'a pas été programmée de façon vectorielle. Pour remédier à cela, on peut utiliser la fonction `Vectorize` ou la fonction `sapply` comme suit.

Console R

```
> g2 = function(x){
+     if(x==0){y=0} else {y=sin(x)/x}
+     return(y)
+ }
> g = Vectorize(g2)
>
> g(0:3)
[1] 0.0000000 0.8414710 0.4546487 0.0470400
```

I. C. GERALDO

Console R

```
> g2 = function(x){
+     if(x==0){y=0} else {y=sin(x)/x}
+     return(y)
+ }
> g = function(x){return(sapply(x,g2))}
>
> g(0:3)
[1] 0.0000000 0.8414710 0.4546487 0.0470400
```

I. C. GERALDO

4.3 EXERCICES

Exercice 4.1 Créer une matrice carrée A d'ordre 7 telle que

$$\begin{cases} a_{ii} = 2 & \text{si} \quad i = 1, \ldots, 7 \\ a_{i,i+1} = 1 & \text{si} \quad i = 1, \ldots, 6 \\ a_{i,i-1} = -1 & \text{si} \quad i = 2, \ldots, 7 \\ a_{ij} = 0 & \text{dans tous les autres cas.} \end{cases}$$

Exercice 4.2 Programmer la fonction factorielle sous forme d'une fonction R récursive (c'est-à-dire qui s'appelle elle-même). Cette fonction sera appelée **fact** et prendra comme seul argument un entier naturel **n**. Elle retournera un message d'erreur lorsque l'argument n'est pas un entier naturel.

Exercice 4.3

1) Ecrire une fonction **matpower** qui prend en arguments une matrice carrée non nulle A ($A \neq 0$) et un entier naturel n et retourne la matrice A^n (on prendra $A^0 = I$ si $A \neq 0$). Un message d'erreur sera affiché si A n'est pas une matrice carrée non nulle ou si n n'est pas un entier naturel.
2) Appliquer la fonction **matpower** à la matrice

$$A = \begin{pmatrix} 2 & 1 & 0 \\ -3 & -1 & 1 \\ 1 & 0 & -1 \end{pmatrix}$$

et vérifier que $A^3 = 0$.

I. C. GERALDO

Exercice 4.4 On s'intéresse à la suite de terme général $S_n = 1 + 2^2 + 3^2 + \cdots + n^2$.

1) Ecrire deux fonctions `f1` et `f2` qui prennent en argument un entier naturel n et calculent S_n de deux façons différentes : l'une avec boucle et l'autre sans aucune boucle.
2) (a) Que font les fonctions suivantes : `Sys.time` ? `difftime` ?
   (b) Comment peut-on les combiner pour mesure le temps d'exécution d'un bloc de commandes ?
3) Pour $n \in \{10,\ 10^2,\ 10^3,\ 10^4,\ 10^5,\ 10^6,\ 10^7\}$, mesurer les temps de calcul (en secondes) de S_n par les deux méthodes. Stocker les résultats sous la forme d'un tableau de données à quatre colonnes correspondant respectivement aux valeurs de n, aux temps d'exécution de `f1` et `f2` et au ratio du temps d'exécution de `f1` sur celui de `f2`.
4) Que peut-on conclure ?

CHAPITRE 5

FONCTIONNALITÉS GRAPHIQUES

Sommaire

5.1 Généralités **80**
5.2 Gestion de la fenêtre graphique **81**
5.2.1 Création et fermeture 81
5.2.2 Fractionnement d'une fenêtre graphique 81
5.2.3 Sauvegarde d'une fenêtre graphique dans un fichier 83
5.3 Fonctions haut niveau **83**
5.3.1 La fonction `plot` 84
5.3.2 Autres fonctions haut niveau 85
5.4 Fonctions bas niveau **85**
5.4.1 Ajout de points, de lignes ou de symboles 86
5.4.2 Ajout de texte 87
5.4.3 Ajout d'un quadrillage sur le graphique 87
5.4.4 Gestion des axes 87
5.5 Fonction `curve` : une fonction à la fois haut niveau et bas niveau **88**
5.6 Exercices **89**

I. C. GERALDO

5.1 GÉNÉRALITÉS

Il existe trois (03) types de commandes graphiques :

- les fonctions **haut niveau** : elles créent une fenêtre graphique s'il n'y en a pas (ou effacent le contenu de la fenêtre graphique s'il y en a déjà une) et y tracent le nouveau graphique ; elles seules sont capables de gérer les caractéristiques de la fenêtre graphique telles que le titre, les axes, etc. . . ;

- les fonctions **bas niveau** : elles sont seulement capables d'ajouter des graphiques, du texte ou des symboles sur un graphique existant créé par une fonction haut niveau ; elles ne peuvent donc être appelées qu'après appel d'une fonction haut niveau ;

- les fonctions **interactives** (non vues dans ce cours) : elles permettent d'interagir sur le graphique afin d'ajouter ou d'en extraire des informations à l'aide de la souris.

5.2 GESTION DE LA FENÊTRE GRAPHIQUE

5.2.1 Création et fermeture

Pour tracer un graphique, on commence par une fonction haut niveau qui va créer la fenêtre graphique et y tracer le graphique tout en gérant les caractéristiques de la fenêtre graphique telles que le titre, les axes, etc.... Ensuite, viennent les fonctions bas niveau. Ainsi, dans la plupart des cas, la fenêtre graphique créée par la fonction haut niveau sera suffisante. Dans quelques rares cas, il peut être nécessaire de créer de nouvelles fenêtres graphiques. Les commandes suivantes permettent de gérer de telles situations.

`x11() ; X11() ; windows(); dev.new()`	Commandes équivalentes permettant d'ouvrir une nouvelle fenêtre graphique vierge sur laquelle il faut nécessairement faire appel en premier à une fonction haut niveau.
`dev.list()`	Affichage d'un vecteur contenant les numéros des fenêtres graphiques ouvertes.
`dev.set(1)`	Commande permettant d'activer la fenêtre graphique numéro 1. C'est sur elle que seront tracés les graphiques jusqu'à sa fermeture ou jusqu'à l'activation d'une autre fenêtre.
`dev.off(1)`	Fermeture de la fenêtre graphique numéro 1.
`dev.off()`	Fermeture de la fenêtre graphique active.
`graphics.off()`	Fermeture de toutes les fenêtres graphiques.

5.2.2 Fractionnement d'une fenêtre graphique

La figure 5.1 (page 82) donne un exemple de fenêtre graphique fractionnée.

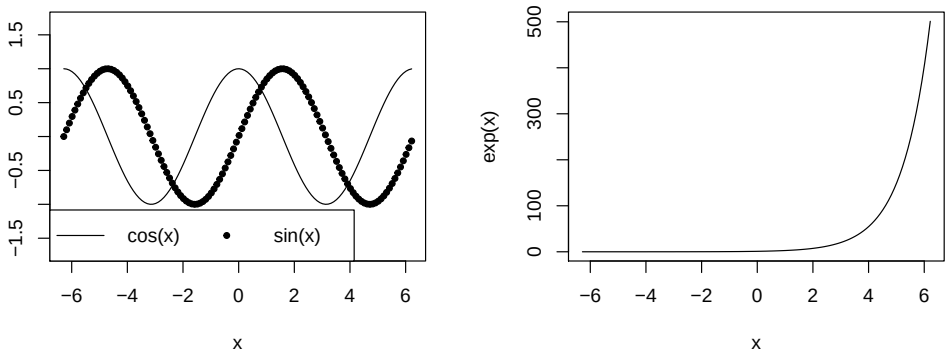


FIGURE 5.1 – Exemple de fenêtre graphique fractionnée

Il existe deux fonctions pour le fractionnement d'une fenêtre graphique.

- On peut utiliser la fonction **`split.screen`** comme suit.

`split.screen(c(2,2))`	Fractionnement de la fenêtre graphique active en 4 sous-fenêtres (2 lignes et 2 colonnes) numérotées par lignes de la gauche vers la droite c'est-à-dire

1	2
3	4

`screen(3)`	Activation du 3ème sous-fenêtre graphique de la fenêtre active (donc celui en ligne 2 et colonne 1).

- On peut utiliser la fonction **`par`** comme suit.

`par(mfrow=c(2,2))`	Fractionnement de la fenêtre graphique active en 4 sous-fenêtres (2 lignes et 2 colonnes) numérotées par lignes de la gauche vers la droite.
`par(mfg=c(2,1))`	Activation de la sous-fenêtre de la ligne 2 et de la colonne 1.

La différence entre les deux fonctions est que lorsqu'on utilise `split.screen`, R ne passe pas à la sous-fenêtre suivante tant que l'on n'a pas fait appel à `screen(...)` alors que lorsqu'on utilise `par(mfrow=c(...))`, R passe à la sous-fenêtre suivante à chaque appel d'une

fonction haut niveau (sauf si l'on sélectionne la sous-fenêtre à activer avec `par(mfg=...)`). Les deux fonctions ne doivent pas être mélangées.

5.2.3 Sauvegarde d'une fenêtre graphique dans un fichier

`dev.size("px")`	Affiche les dimensions en pixels de la fenêtre graphique active (1 px = 0.02646 cm). On peut remplacer l'unité `"px"` par `"cm"` ou `"in"`.
`dev.print(png, "test.png",` `  width=700, height=600)`	Sauvegarde dans le répertoire courant la fenêtre graphique active dans le fichier "test.png" avec les paramètres spécifiés. Il faut préciser les dimensions (l'unité est le pixel). Les dimensions doivent être suffisamment grandes car 1 px = 0.02646 cm. Le paramètre `png` et le nom du fichier peut être respectivement remplacé par `jpeg` et "test.jpg").

`dev.print(pdf, "test.pdf",`
`  width=7, height=5)`
 	
`dev.copy2pdf(file="test.pdf",`
`  width=7, height=5)`
	Commandes équivalentes permettant de sauvegarder dans le répertoire courant la fenêtre graphique active dans le fichier PDF nommé "test.pdf" avec les dimensions en pouces (1 pouce = 2.54cm).

5.3 Fonctions haut niveau

Comme indiqué au début du chapitre, elles créent une fenêtre graphique s'il n'y en a pas (ou effacent le contenu de la fenêtre graphique s'il y en a déjà une) et y tracent le nouveau graphique ; elles seules sont capables de gérer les caractéristiques de la fenêtre graphique telles que le titre, les axes, etc.... Lorsqu'on fait appel deux fois consécutivement à une fonction haut niveau sur une même fenêtre graphique non fractionnée avec `par(mfrow=...)`, le deuxième appel efface le graphique tracé par le premier.

5.3.1 La fonction `plot`

`plot(x)`	Si `x` est un vecteur de nombres réels de n composantes, cette commande produit le nuage de points (i, x_i) avec $i = 1, \ldots, n$ en abscisses et les composantes de `x` en ordonnées.
`plot(x,y)`	Si `x` et `y` sont des vecteurs numériques de même longueur, cette commande produit le nuage de points (x_i, y_i) avec les composantes de `x` en abscisses et celles de `y` en ordonnées.
`plot(M)`	Si `M` est une matrice à deux colonnes, cette commande produit un nuage de points avec les composantes de la première colonne en abscisses et celles de la $2^{\text{ème}}$ colonne en ordonnées.
`x=seq(-2,5,0.1)` `plot(x,exp(x))`	Représentation graphique de la fonction exponentielle à l'aide d'un nuage de points sur l'intervalle $[-2, 5]$.

Amélioration. Par défaut, la commande `plot` représente un nuage de points où chaque point est représenté par un cercle. Il est possible d'améliorer l'aspect du graphique en spécifiant la valeur de certains paramètres.

`plot(x,y,type="p",pch="+")`	Affichage du nuage de points où les points sont représentés avec le symbole "+" (`pch` = point character).
`plot(x,y,type="p",pch=20)`	Le `pch` peut aussi être donné par un nombre entre 0 et 25. Voir la table 3.1 (page 65) pour la correspondance entre les valeurs et les symboles.
`plot(x,y,type="l")`	Les points ne sont pas affichés distinctement mais ils sont reliés par des lignes.
`plot(x,y,type="o")`	Nuage de points représentés par des cercles et reliés par des lignes.
`plot(x,y,type="n")`	Le graphique ne s'affiche pas mais la fenêtre graphique est créée et peut être paramétrée notamment dans le but d'utiliser les fonctions bas niveau.

`plot(x, y, axes=FALSE)`	Graphique sans axes.
`plot(x, y, main="Titre", xlab="...", ylab="...")`	Les paramètres `main`, `xlab` et `ylab` permettent respectivement d'indiquer le titre du graphique, le nom de l'axe des abscisses et celui de l'axe des ordonnées.
`plot(x, y, col="blue")`	Gestion de la couleur. **N.B.** On rappelle que la commande `colors()` permet d'afficher la liste des couleurs disponibles.
`plot(x, y, xlim=c(-1,3), ylim=c(0,2))`	Les paramètres `xlim` et `ylim` permettent de délimiter l'axe des abscisses et l'axe des ordonnées.
`plot(x, y, lwd=2)`	Le paramètre `lwd` (line width) permet de gérer l'épaisseur des traits.
`help("par")`	Consulter la documentation de R pour plus de détails sur les paramètres graphiques.

5.3.2 Autres fonctions haut niveau

Il existe d'autres fonctions haut niveau dont nous avons déjà présenté certaines dans le chapitre 3 portant sur la statistique descriptive. Ce sont les fonctions `barplot`, `boxplot`, `hist` et `pie`.

5.4 FONCTIONS BAS NIVEAU

Comme indiqué en début du chapitre, elles sont seulement capables d'ajouter des graphiques, du texte ou des symboles sur un graphique existant créé par une fonction haut niveau. Elles ne peuvent donc être appelées qu'après appel d'une fonction haut niveau.

5.4.1 Ajout de points, de lignes ou de symboles

`points(x,y,pch=20)`	Ajout sur le graphique d'un nuage de points où les points sont représentés par le symbole correspondant à `pch=20` (voir la table 3.1, page 65). Commande similaire à `plot(x,y, type="p", pch=20)` mais sans pouvoir effacer le graphique existant ni modifier les caractéristiques telles que les axes et le titre.
`lines(x,y)`	Commande similaire à `plot(x,y, type="l")` mais sans effacer le graphique existant ni modifier les caractéristiques telles que les axes et le titre.
`lines(x,y,col="blue",lty="dotted")`	Spécification de la couleur (bleue) et du type de ligne (pointillés). L'argument `lty` signifie « line type ».

`abline(2,-3)`
`abline(b=2,a=-3)`
`abline(a=-3,b=2)`
	Commandes équivalentes permettant d'ajouter la droite d'équation $y = a + bx$ sur la fenêtre graphique active.

`abline(v=1,h=3)`	Commande permettant d'ajouter la droite verticale d'équation $x = 1$ et la droite horizontale d'équation $y = 3$ sur la fenêtre graphique active.
`segments(x0, y0, x1, y1)`	Trace une ligne du point `(x0,y0)` au point `(x1,y1)`.
`arrows(x0, y0, x1, y1)`	Trace une flèche du point `(x0,y0)` au point `(x1,y1)`.
`rect(xleft, ybottom, xright, ytop)`	Ajoute sur le graphique, un rectangle d'extrémités `(xleft,ybottom)` en bas à gauche et `(xright,ytop)` en haut à droite.

5.4.2 Ajout de texte

`title("Titre du graphique")`	Commande permettant d'écrire le titre du graphique. N.B. : C'est une fonction bas niveau donc si le titre a déjà été écrit grâce à l'argument `main` de la fonction haut niveau ayant créé la fenêtre graphique (`plot` par exemple), `title` n'effacera pas l'ancien titre mais écrira dessus.

`legend(x="topright",`
`  legend=c("...", "..."),`
`  col=c("blue", "red"),`
`  lty=c("solid", "dotted"))`
	Commande permettant d'ajouter sur la fenêtre graphique active, une légende placée en haut à droite avec deux libellés ... et ..., le premier associé à une ligne continue bleue et le second à une ligne pointillée rouge.

`text(x,y,texte)`	Si `x` et `y` sont des vecteurs numériques réels de longueur n, cette commande permet d'écrire sur le graphique à chaque position $(x_i, y_i), i =, \ldots, n$, la $i^{\text{ème}}$ composante du vecteur de chaînes de caractères `texte`.
`help(plotmath)`	Afficher l'aide sur l'ajout de symboles mathématiques (racine carrée, exposants, etc...) sur un graphique.

5.4.3 Ajout d'un quadrillage sur le graphique

`grid()`	Commande permettant d'ajouter un quadrillage sur le graphique.
`help(grid)`	Affichage de la documentation de la fonction `grid` pour plus de détails sur son paramétrage.

5.4.4 Gestion des axes

Les apparences respectives des axes des abscisses et des ordonnées peuvent être réglées grâce à la fonction `axis`. Le lecteur pourra consulter la documentation pour plus de détails.

5.5 FONCTION `curve` : UNE FONCTION À LA FOIS HAUT NIVEAU ET BAS NIVEAU

Par défaut, `curve` est une fonction haut niveau. Elle agit un peu comme `plot`.

```
curve(f, from=-2, to=3)
curve(f,-2,3)
curve(f(x),-2,3)
```

Commandes équivalentes permettant de représenter graphiquement la fonction `f` sur l'intervalle $[-2, 3]$.

N.B. :

- La fonction `f` doit être **vectorisée** (ou **vectorielle**).
- Dans la troisième commande, l'on doit toujours désigner la variable par `x` même si elle porte un autre nom dans la définition de la fonction `f`. Cela permet à R de comprendre qu'il s'agit d'une expression.

```
curve(f, from=-2, to=3, n=200)
```

L'option `n=200` précise le nombre de points à utiliser dans la discrétisation de l'intervalle $[-2, 3]$.

```
curve(f, -2, 3, col="...",
      lwd="...", lty="...",
      xlim="...", main="...")
```

Toutes les options déjà vues plus haut sont valables pour la fonction `curve`.

```
curve(f(2*x), -2, 3)
```

Représentation de la fonction $x \mapsto \texttt{f}(2x)$ sur l'intervalle $[-2, 3]$.

Comme dit plus haut, par défaut `curve` est une fonction haut niveau. Donc, si on l'utilise deux fois (sauf sur un graphique fractionné avec `par(mfrow=...)`), le deuxième `curve` effacera le graphique tracé par le premier. Pour indiquer que la fonction `curve` doit être utilisée comme une fonction bas niveau, il faut ajouter l'option `add=TRUE`.

```
curve(..., add=TRUE)
```

L'option `add=TRUE` indique de ne pas effacer le graphique existant.

5.6 EXERCICES

Exercice 5.1 Représenter chacune des fonctions suivantes sur l'intervalle mentionné :

$$f : x \longmapsto \mathrm{e}^{x}, \qquad I = [-3, 4];$$

$$g : x \longmapsto \frac{1}{\sqrt{2\pi}} \mathrm{e}^{-\frac{x^2}{2}}, \qquad I = [-4, 4];$$

$$h : x \longmapsto \log(x), \qquad I =]0, 10].$$

Pour chaque graphique, veiller à renseigner les noms des axes ainsi que le titre (la fonction g correspond à la densité de la loi normale $\mathcal{N}(0, 1)$).

Exercice 5.2 Représenter les fonctions suivantes sur l'intervalle $[-3, 3]$:

$$f(x) = \begin{cases} 9/4 & \text{si} \quad x < -3/2 \\ x^2 & \text{si} \quad -3/2 \leqslant x < 1 \\ 1 & \text{si} \quad x \geqslant 1 \end{cases} \quad ; \quad g(x) = \begin{cases} 1 & \text{si} \quad x < -1 \\ x + 2 & \text{si} \quad -1 \leqslant x < 0 \\ 2 - x & \text{si} \quad 0 \leqslant x < 2 \\ 0 & \text{si} \quad x \geqslant 2 \end{cases}$$

Exercice 5.3 Représenter les fonctions sinus et cosinus sur $[-2\pi, 2\pi]$ superposées sur le même graphique à la manière de la figure 5.2.

GERALDO

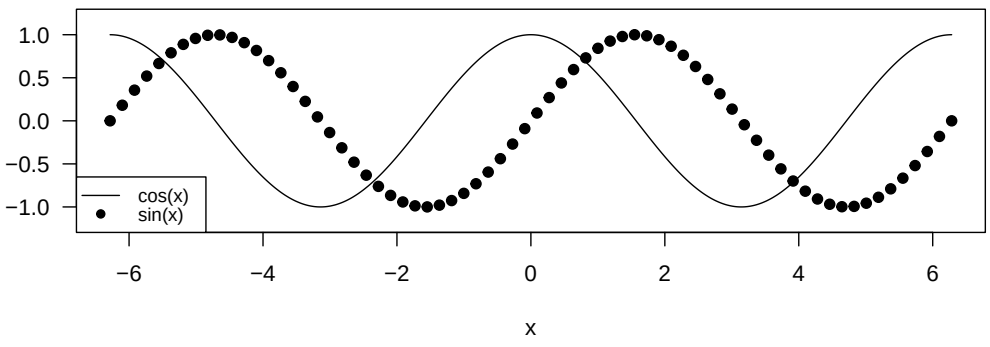


FIGURE 5.2 – Représentation des fonctions sinus et cosinus sur $[-2\pi, 2\pi]$.

Exercice 5.4 Tracer les courbes paramétrées suivantes pour $t \in [-10, 10]$:

1) $\begin{cases} x(t) = \cos^2(t) \\ y(t) = \cos^3(t)\sin(t) \end{cases}$

2) $\begin{cases} x(t) = \dfrac{t}{1+t^4} \\ y(t) = \dfrac{t^3}{1+t^4} \end{cases}$

3) $\begin{cases} x(t) = \sin^3(t) \\ y(t) = \cos(t) - \cos^4(t) \end{cases}$

4) $\begin{cases} x(t) = (250 + 220\sin(6t))\cos(t) \\ y(t) = (250 + 220\sin(6t))\sin(t) \end{cases}$

Exercice 5.5 Représenter sur un même graphique les fonctions $x \mapsto x^2$ et $x \mapsto \sqrt{x}$ sur les intervalles $[0, 2]$ et $[0, 4]$ respectivement. Ajouter au graphique obtenu, la première bissectrice en pointillés pour apprécier la symétrie des deux courbes par rapport à cette dernière. Le résultat obtenu sera semblable à la figure 5.3.

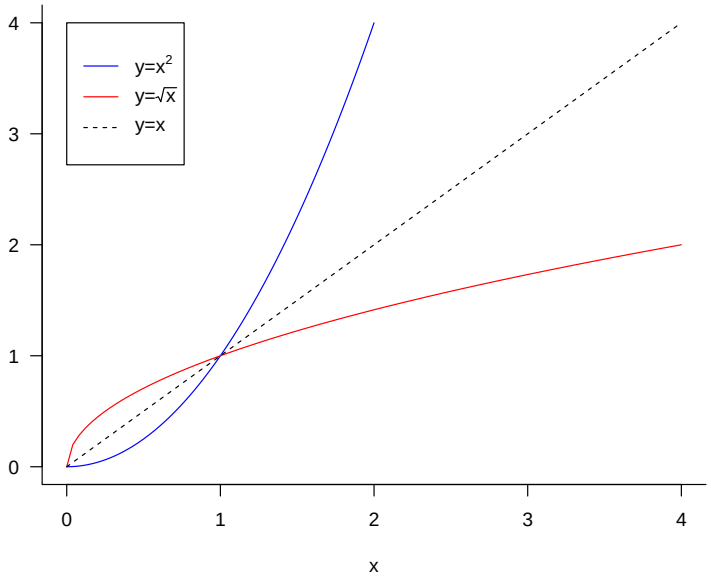


FIGURE 5.3

CHAPITRE 6

PROBABILITÉS, INTÉGRALES ET SIMULATIONS

Sommaire

6.1 **Lois de probabilités usuelles** 91
6.2 **Tirages avec ou sans remise** 92
6.3 **Réplication d'une commande** 93
6.4 **Calcul d'intégrales** 93
6.5 **Exercices** 95

6.1 LOIS DE PROBABILITÉS USUELLES

Toutes les lois de probabilités usuelles sont disponibles dans R. Chaque loi est identifiée par une abréviation (voir la table 6.1).

TABLE 6.1 – Liste des lois de probabilités usuelles

Loi	Abréviation
Beta	`beta`
Binomiale (y compris Bernoulli)	`binom`
Binomiale négative	`nbinom`
Cauchy	`cauchy`
Exponentielle	`exp`
Fisher-Snedecor	`f`
Gamma	`gamma`
Géométrique	`geom`
Hypergéométrique	`hyper`
Khi-deux	`chisq`
Log-normale	`lnorm`
Logistique	`logis`
Normale	`norm`
Poisson	`pois`
Student	`t`
Uniforme	`unif`
Weibull	`weibull`

I. C. GERALDO

Pour chaque loi d'abréviation `xxx`, quatre (04) fonctions sont disponibles, identifiées par un préfixe :

`dxxx`	Fonction densité de la loi `xxx`.
`pxxx`	Fonction de répartition de la loi `xxx`.
`qxxx`	Fonction quantile de la loi `xxx`.
`rxxx`	Génération de nombres aléatoires suivant la loi `xxx`.

Le lecteur est invité à consulter l'aide pour plus de détails sur les paramètres des différentes lois. Nous donnons ci-dessous un exemple de loi discrète et un exemple de loi continue.

`dbinom(3,20,1/2)`	Probabilité individuelle de la loi binomiale $\mathcal{B}(20, \frac{1}{2})$ évaluée en 3 c'est-à-dire $\mathbb{P}(X = 3)$ lorsque $X \rightsquigarrow \mathcal{B}(20, \frac{1}{2})$.
`pbinom(3,20,1/2)`	Fonction de répartition de la loi $\mathcal{B}(20, \frac{1}{2})$ évaluée en 3 c'est-à-dire $\mathbb{P}(X \leqslant 3)$.
`qbinom(0.9,20,1/2)`	Quantile d'ordre 0.9 de la loi $\mathcal{B}(20, \frac{1}{2})$.
`rbinom(10,20,1/2)`	Simulation d'un vecteur de 10 réalisations indépendantes de la loi $\mathcal{B}(20, \frac{1}{2})$.
`dnorm(1,0,2)`	Densité de la loi normale $\mathcal{N}(0,2)$ évaluée en 1.
`pnorm(1,0,2)`	Fonction de répartition de la loi $\mathcal{N}(0,2)$ évaluée en 1.
`qnorm(0.9,0,2)`	Quantile d'ordre 0.9 de la loi $\mathcal{N}(0,2)$.
`rnorm(10,0,2)`	Simulation d'un vecteur de 10 réalisations indépendantes de la loi $\mathcal{N}(0,2)$.

6.2 TIRAGES AVEC OU SANS REMISE

`sample(1:10,100,replace=TRUE)`	Tirage avec remise de 100 éléments parmi les nombres $1, \ldots, 10$. L'option `replace=TRUE` précise que le tirage se fait avec remise. Par défaut (si rien n'est précisé), `replace` vaut `FALSE`.

`sample(1:10); sample(10)`	Commandes équivalentes permettant de faire une permutation aléatoire des entiers naturels 1, 2, . . ., 10.
`sample(c("A","B","C"), size=5, replace=TRUE, prob=c(0.2,0.5,0.3))`	Tirage avec remise de 5 lettres parmi les lettres A, B et C. Si l'on ne veut pas un tirage équiprobable, on précise les probabilités de tirage dans le vecteur `prob`.

6.3 Réplication d'une commande

`replicate(n,expr)`	Exécute `n` fois le code `expr` et stocke les résultats sous forme de vecteur (si `expr` renvoie un nombre) et sous forme de matrice (si `expr` renvoie un vecteur).

La fonction `replicate` permet d'éviter l'usage de boucles. Par exemple, le code suivant simule 5 fois la génération de 1 000 valeurs suivant la loi normale centrée réduite et calcule la moyenne à chaque fois.

Console R

```
> replicate(5,mean(rnorm(1000)))
[1] -0.01514964 -0.04786793 -0.06151126 -0.07324169  0.01370721
```

6.4 Calcul d'intégrales

`integrate(f,a,b,...)`	Calcul de l'intégrale définie de la fonction `f` de `a` à `b`. Le symbole . . . représente les arguments supplémentaires passés à la fonction `f`. N.B. : `f` doit être une fonction vectorielle.

Exemple 6.1 Voici le code R pour vérifier l'intégrale de Gauss suivante :

$$\int_{-\infty}^{+\infty} e^{-\frac{x^2}{2}}\, dx = \sqrt{2\pi}. \tag{6.1}$$

Console R

```
> f=function(x){exp(-x^2/2)}
> integrate(f,-Inf,Inf)
2.506628 with absolute error < 0.00023
>
> # On peut aussi faire
```

I. C. GERALDO

```
> integrate(function(x){exp(-x^2/2)},-Inf,Inf)
2.506628 with absolute error < 0.00023
>
> sqrt(2*pi)
[1] 2.506628
```

Remarque 6.1 Même si à l'affichage, on obtient la valeur de l'intégrale, il faut noter que la fonction `integrate` ne retourne pas une valeur réelle mais plutôt une liste particulière de classe "integrate" ayant cinq (5) composantes comme suit :

Console R

```
> y=integrate(function(x){exp(-x^2/2)},-Inf,Inf)
> print(y)
2.506628 with absolute error < 0.00023
> names(y)
[1] "value"        "abs.error"    "subdivisions" "message"
[5] "call"
> print(y$value)
[1] 2.506628
```

Pour récupérer la valeur de l'intégrale, il faut donc utiliser `y$value`.

I. C. GERALDO

6.5 EXERCICES

Exercice 6.1 Soit X une variable aléatoire de loi binomiale de paramètres $n = 50$ et $p = 1/3$. Calculer les probabilités suivantes :

$$\mathbb{P}(X = 1) \quad ; \quad \mathbb{P}(X \leqslant 5) \quad ; \quad \mathbb{P}(X \geqslant 15) \quad ; \quad \mathbb{P}(X \notin \{3, 4, 10, 15\}) \quad ; \quad \mathbb{P}(X \in 2\mathbb{N}).$$

Exercice 6.2 Soit X une variable aléatoire de loi normale standard $\mathcal{N}(0, 1)$.

1) Calculer les probabilités suivantes : $\mathbb{P}(X \leqslant 1)$, $\mathbb{P}(X \geqslant 2.6)$ et $\mathbb{P}(0.5 < X \leqslant 1)$.
2) Calculer les quantiles d'ordre 0.95 et 0.975.
3) Représenter graphiquement la densité et la fonction de répartition de la loi de X.

Exercice 6.3 Soit X une variable suivant la loi normale $\mathcal{N}(3, 2)$. Calculer les probabilités suivantes : $\mathbb{P}(X < 4)$, $\mathbb{P}(X < -1)$, $\mathbb{P}(X > 1)$ et $\mathbb{P}(-3 < X < 4)$.

Exercice 6.4

1) Créer un vecteur x de $n = 1000$ réalisations aléatoires de la loi normale centrée réduite.
2) Faire l'histogramme des données contenues dans ce vecteur et y superposer la fonction densité de la loi normale centrée réduite. Que remarque-t-on ?

Exercice 6.5 Ecrire le code R permettant d'obtenir exactement la figure 6.1 représentant la densité de la loi $\mathcal{N}(m, \sigma)$ pour différentes valeurs des paramètres m et σ.

GERALDO

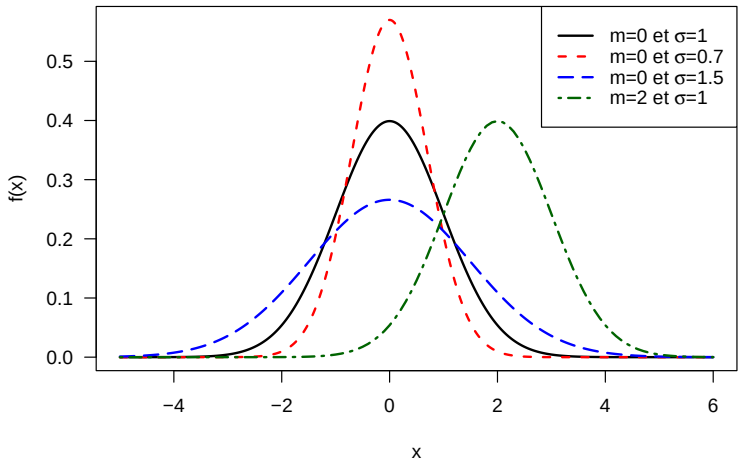


FIGURE 6.1 – Représentation de la densité de la loi $\mathcal{N}(m, \sigma)$ pour différentes valeurs de m et σ.

Exercice 6.6

1) En utilisant la fonction `sample` et le vecteur `piece=c("F","P")` (dont les composantes représentent FACE et PILE), simuler $n = 5\,000$ lancers d'une pièce de monnaie équilibrée.
2) Calculer la proportion p de `F` (elle devrait être proche de 0.5).

Exercice 6.7 Un ami vous propose le jeu suivant. Vous lancez deux (2) fois un dé supposé parfaitement équilibré dont les faces sont numérotées de 1 à 6. A chaque lancer, si le résultat est 5 ou 6, vous gagnez 3 000 FCFA ; si le résultat est 4, vous gagnez 1 000 FCFA et si c'est 3 ou moins, vous perdez 2 500 FCFA.

1) Ecrire une fonction `jeu` permettant de simuler le jeu.
2) Simuler ce jeu 1 000 fois.
3) Résumer les résultats sous forme de loi de probabilité en utilisant un tableau de contingence et représenter cette loi par un diagramme en bâtons.
4) Ce jeu serait-il avantageux pour vous ? Justifier la réponse.

I. C. GERALDO

RÉFÉRENCES BIBLIOGRAPHIQUES

[1] Bertrand, F. & Maumy-Bertrand, M. (2018). *Initiation à la Statistique avec R*. Paris : Dunod, 3ème edition.

[2] Biernacki, C. (2010). Logiciel R - Tutorial avec applications statistiques. Notes de cours, Université Lille 1.

[3] Broc, G. (2016). *Stats faciles avec R*. De Boeck Superieur.

[4] INSEED – Togo (2022). Annuaire statistique national 2022. URL https://inseed.tg/download/5461/?tmstv=1729030151.

[5] Lafaye de Micheaux, P., Drouilhet, R., & Liquet, B. (2014). *Le logiciel R : Maîtriser le langage - Effectuer des analyses statistiques*. Springer, 2ème edition.

[6] Paradis, E. (2005). R pour les débutants. URL http://cran.r-project.org/doc/contrib/Paradis-rdebuts_fr.pdf.

[7] R Core Team (2022). *R : A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria. https://www.R-project.org/.

[8] Venables, W. N., Smith, D. M., & the R Core Team (2014). An Introduction to R. Version 3.0.3 (2014-03-06).

I. C. GERALDO